

NEURAL ARCHITECTURES



Instructor:
Pradeep Kadubandi

Introduction – Pradeep

- **Software Engineer for a decade before switching into ML Engineer (including getting a Masters degree). Currently @ Apple. Have been in ML space for the last 5 years.**
- **Past experience: ex-Meta, ex-Microsoft**
- **Experiences: Computer Vision, LLM-based systems, Distributed Systems**

**Welcome to today's class, before we begin,
pop into the chat:**

Your Name

Role

Company

Where you're based

Optimize Your Experience

1. Interact with your instructors via Sunday live class.
2. Don't be shy to speak up and get clarifications !
3. Solve your assignments, MCQs and other assessments and get feedback (Thursday review session)
4. Use your resources! It's your experience – what you put in is what you'll get out.

Context for this class

	ML-15: Intro to Neural Network		⋮
	ML-16: DL Data Structures and Frameworks	Jun 29, 2023	⋮
	ML-17 :Neural Architectures	11:37 AM	⋮
	ML-18: Computer Vision 1		⋮
	ML-19: Computer Vision 2		⋮
	ML-20: SQL		⋮
	ML-21: Modern ML Architectures		⋮
	ML-22: NLP-1		⋮
	ML-23: NLP-2		⋮
	ML-24: DL Mini Project		⋮
	ML-25: Generative AI		⋮

Today's Agenda

	1st Part (9-10.20 PT)	2nd Part (10.20-12 PT)	3rd Part (12-1.15 PT)	4th Part (1.15-1.30 PT)
0:00 - 0:40	Review + Introduction to Deep Learning + FNN	CNN model	RNN model	Miscellaneous topic (15 minutes)
0:40 - 1:00	Feed Forward Neural network model coding practice	CNN coding practice	RNN coding practice	
1:00 - 1:15	Q&A	Q&A	Q&A	
1:15 - 1:20	Break	Break (25 minutes)		

Can you Answer these?

How do you think Youtube generates subtitles for the videos?

OR

How does Iphone unlock based on faceID?

OR

How does Google Translate work for so many languages?

Can you give me some more examples from real-life where you think deep learning might be used?

Intro to Deep Learning (Recap)

Learning Objective

Understand various neural network based architectures and the fundamentals behind them

Section Agenda

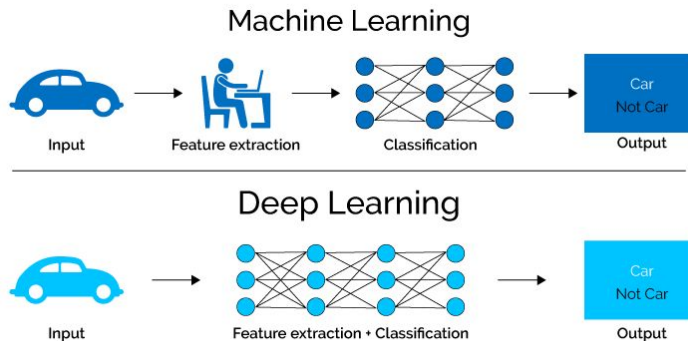
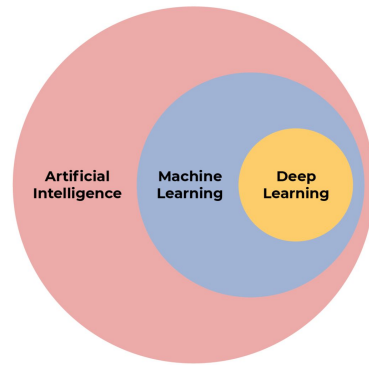
- ❑ Introduction to deep Learning
- ❑ Ways to identify deep Learning Problems
- ❑ Deep Learning Vs Neural Network
- ❑ Introduction to Deep Learning Models
- ❑ Real-life application of deep Learning
- ❑ Advantages of deep learning
- ❑ Challenges with deep Learning
- ❑ Quick recap of Loss Function, Activation Function and Gradient Descent Algorithm

What is Deep Learning - Answer

Deep learning is a branch of machine learning that uses artificial neural networks to teach computers how to learn and make decisions by themselves from data without relying on predefined rules or explicit feature engineering.

It's used for tasks like

- recognizing patterns in images
- understanding speech
- Autonomous vehicles, etc

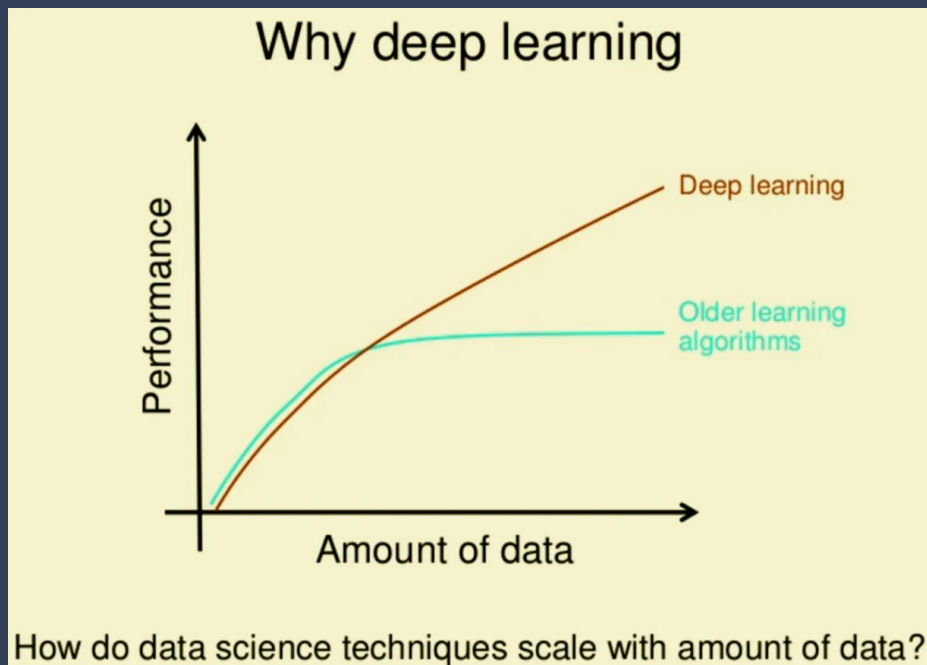


How to identify deep learning problems?

Large amount of high-dimensional, unstructured data, and you require the model to automatically learn intricate patterns and representations from the data

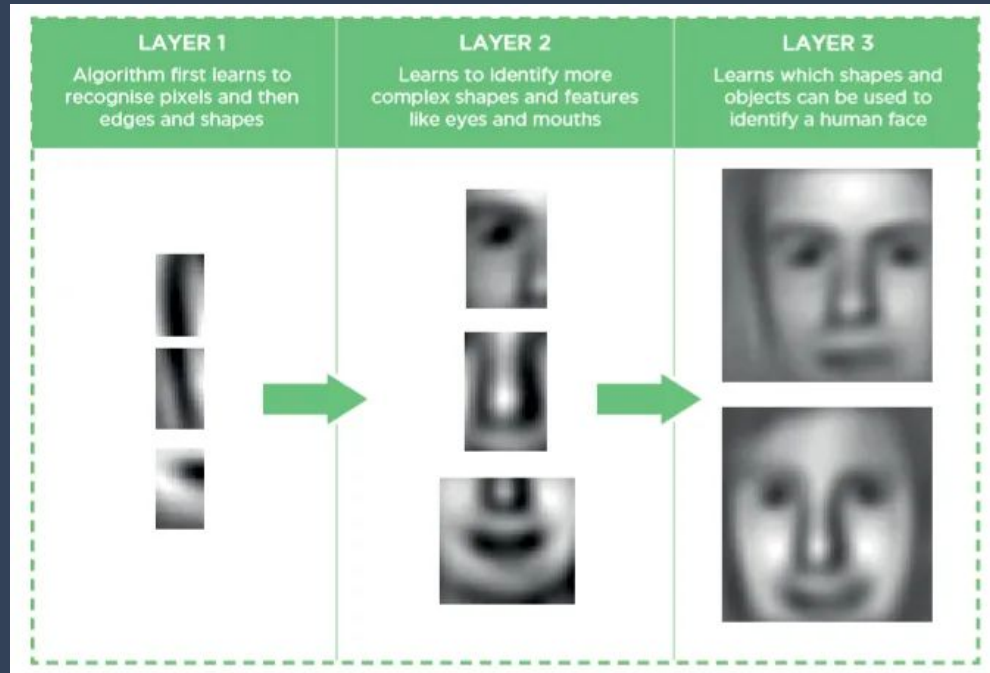
How to identify deep learning problems - Answer

1. *Unstructured or large amount of data*



How to identify deep learning problems - Answer

2. *Complex patterns or representations*

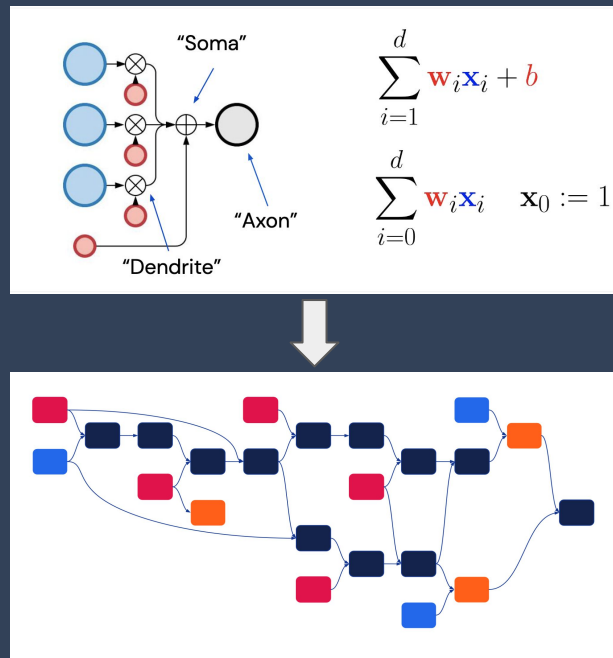


How to identify deep learning problems

Some other considerations

- Lack of domain-specific feature engineering
- Availability of computational resources
- Existing benchmarks and research

Deep Learning Vs Neural Network



Deep learning involves the use of neural networks with many layers, making it more complex and capable of handling intricate patterns in data.

Why Deep Learning?

- **Learning Complex Patterns**

- Models like Convolutional Neural Networks (CNNs) have learned to recognize complex patterns in images, including objects, animals, and scenes

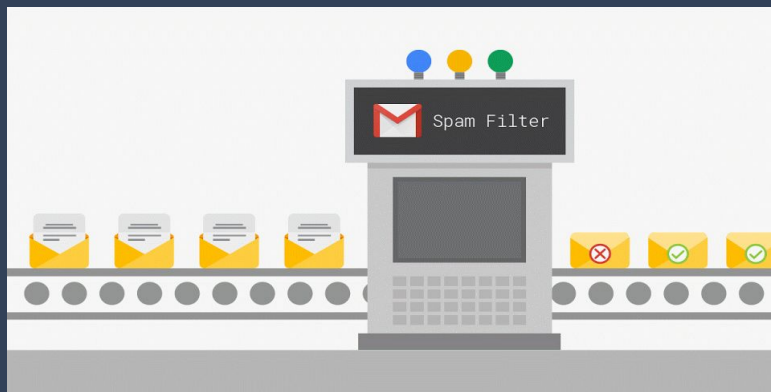
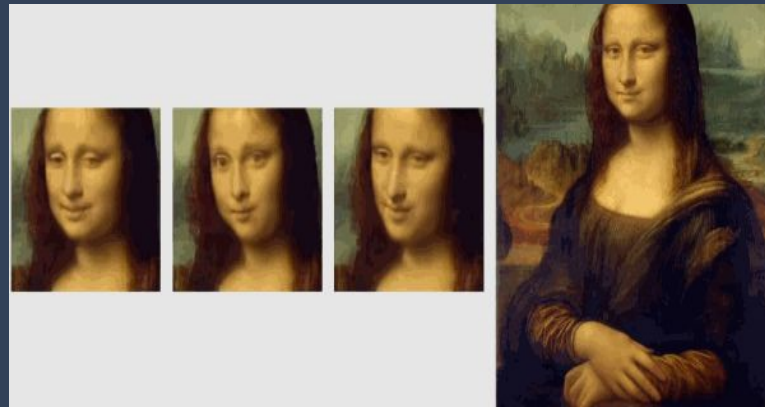
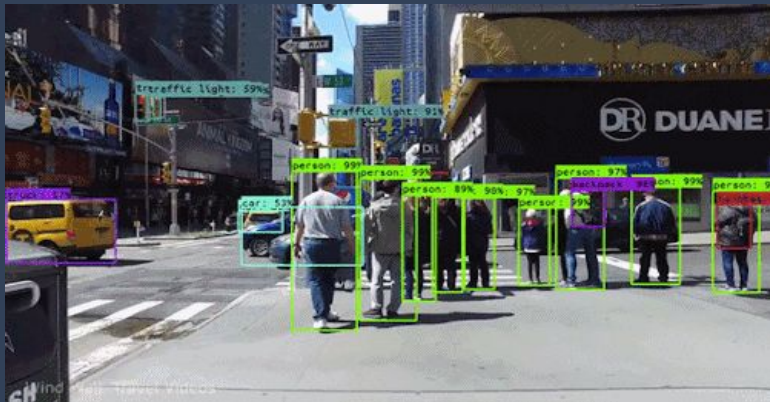
- **Transfer Learning and Pretrained Models**

- The pre-trained transformer based BERT/GPT model captures a rich understanding of language, making it a valuable starting point for various NLP tasks

- **Versatility and Adaptability**

- Deep learning can handle multi-modal data, combining information from multiple sources (e.g., text, images, and audio) to perform tasks like multimedia content understanding or sentiment analysis in social media posts

Real-life application of deep learning



Introduction to deep learning models

- Feed Forward Neural Networks (FNNs)
- Convolutional Neural Networks (CNNs)
- Recurrent Neural Networks (RNNs)

We would be looking into these models in-depth further in the class.

Loss Functions - Quick Recap

Loss functions are used to quantify the discrepancy between the predicted output of the network and the target output

- Mean Squared Error (MSE) Loss

$$MSE = \frac{1}{n} \sum \underbrace{\left(y - \hat{y} \right)^2}_{\substack{\text{The square of the difference} \\ \text{between actual and} \\ \text{predicted}}}$$

- Binary Cross-Entropy Loss

$$H_p(q) = -\frac{1}{N} \sum_{i=1}^N y_i \cdot \log(p(y_i)) + (1 - y_i) \cdot \log(1 - p(y_i))$$

- Categorical Cross-Entropy Loss

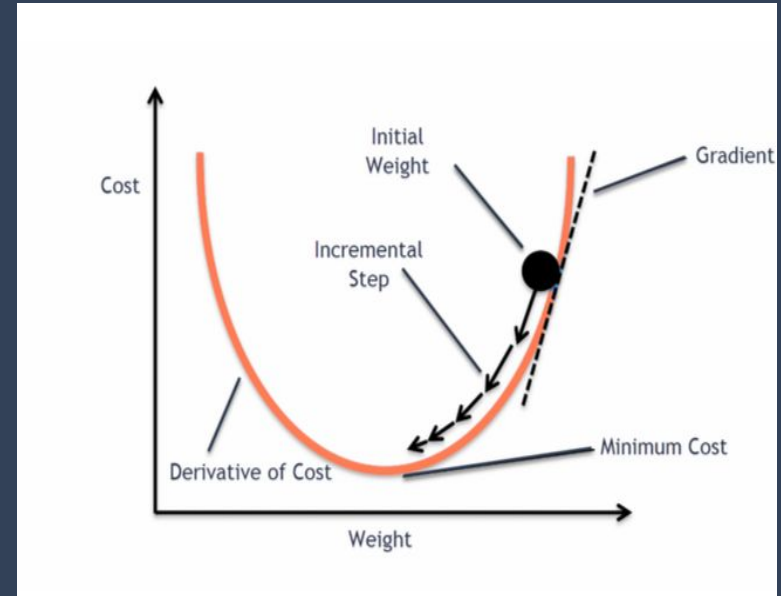
$$\text{Loss} = - \sum_{i=1}^{\text{output size}} y_i \cdot \log \hat{y}_i$$

Gradient Descent Algorithm - Quick Recap

- The gradient descent algorithm is an optimization algorithm commonly used to train a model
- It iteratively updates the network's weights and biases based on the gradients of the loss function with respect to the parameters

Gradient Descent Algorithm - Quick Recap

1. Initialize Weights and Biases
2. Forward Propagation
3. Calculate Loss
4. Backpropagation
5. Update Weights and Biases
6. Repeat Steps 2-5
7. Training Completion

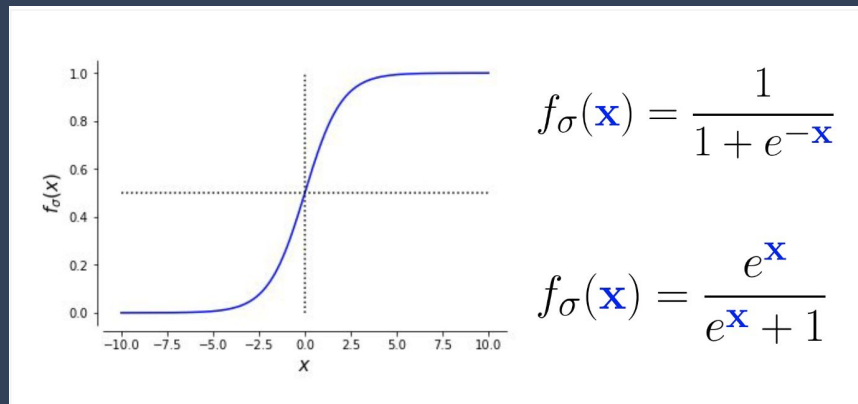


Activation Functions - Quick Recap

- Activation functions are often called non-linearities.
- Activation functions are applied point-wise.

Examples

- Sigmoid
- Hyperbolic Tangent (Tanh)
- Rectified Linear Unit (ReLU) + (GELU*)
- Softmax



Sigmoid

Section Summary

- ❏ What is deep learning?
- ❏ Ways to identify deep learning problems
- ❏ Deep Learning vs Neural Network
- ❏ Deep Learning Models
- ❏ Real-life application of deep learning
- ❏ Advantages of deep learning
- ❏ Challenges with deep learning
- ❏ Loss Functions
- ❏ Gradient Descent Algorithm
- ❏ Activation Functions

Learning Objectives

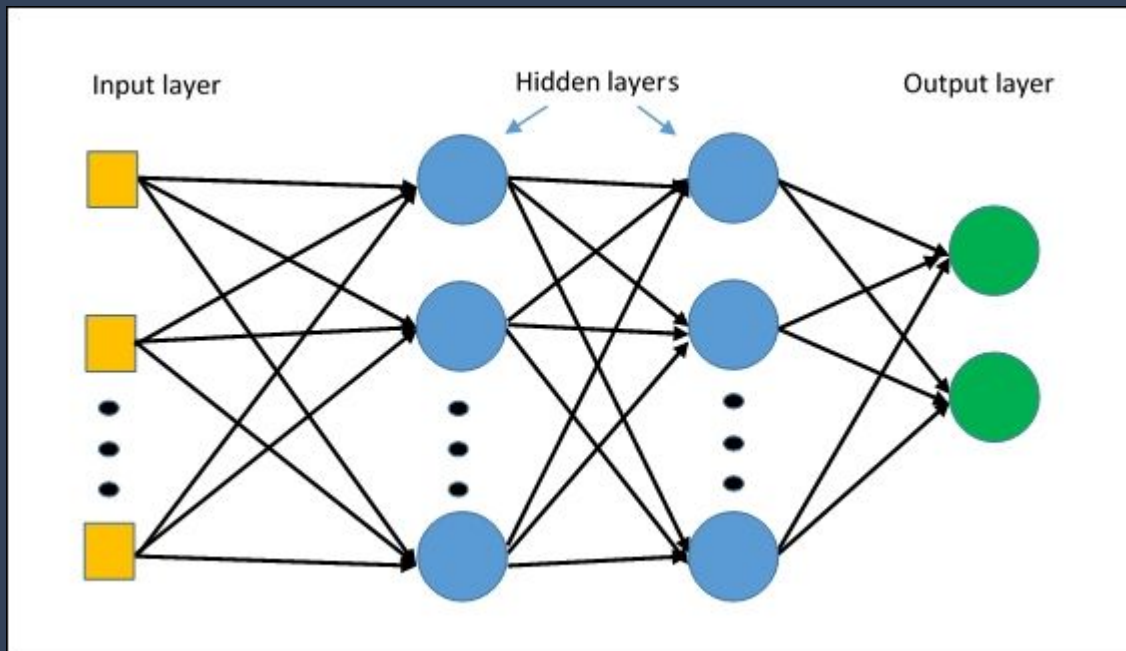
Developing an understanding of FNN (ML) working, principle and its applications

Section Agenda

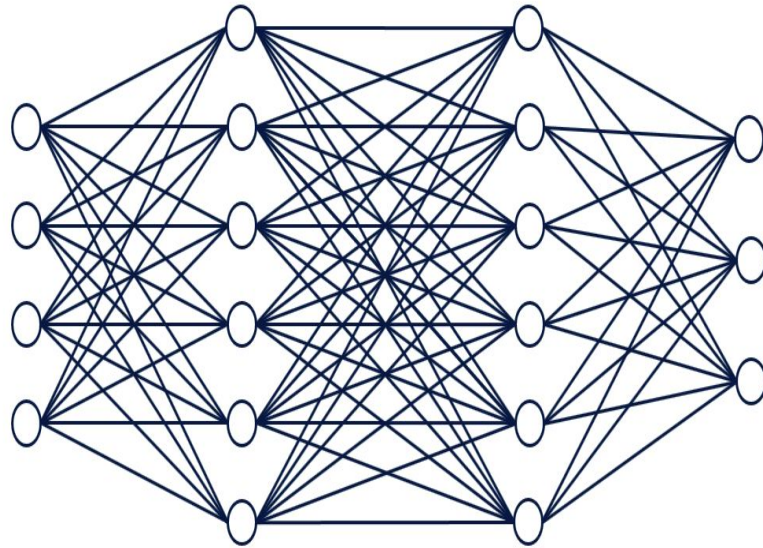
- ❑ Introduction to FNN
- ❑ Principle of FNN
- ❑ FNN Layers
- ❑ Types of FNN Models
- ❑ Advantages of FNN
- ❑ Limitations of FNN
- ❑ Applications of FNN
- ❑ Example of FNN on Dataset

What is FNN?

FNN stands for **F**eedforward **N**eural **N**etwork, which is a type of artificial neural network. It is also commonly referred to as a multi-layer perceptron (MLP).



FNN Data Flow

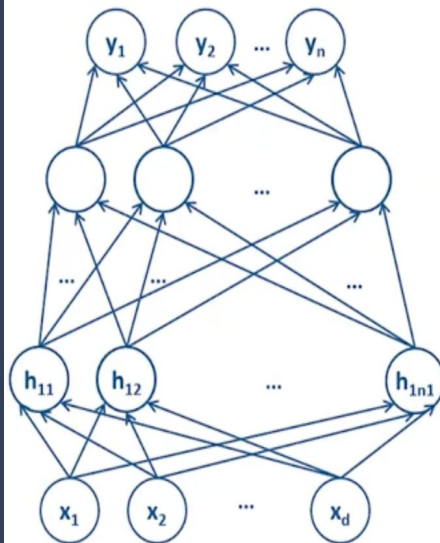


Layers of FNN

A Feedforward Neural Network (FNN) typically consists of three main types of layers:

- **Input Layer:** Represents dimension of input vector
- **Hidden Layers:** Represents intermediary node that divide input space into regions with (soft) boundaries
- **Output Layer:** Final layer that produces output or prediction

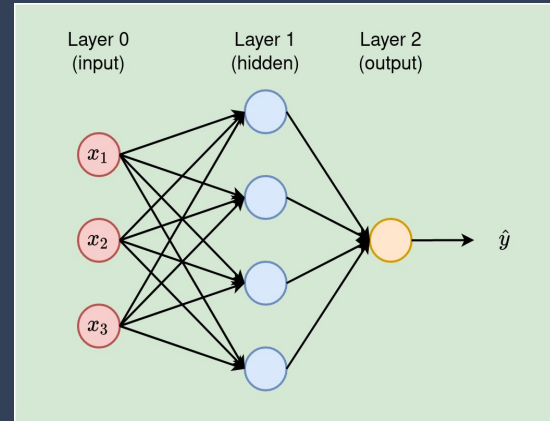
Basic structure of a neural network



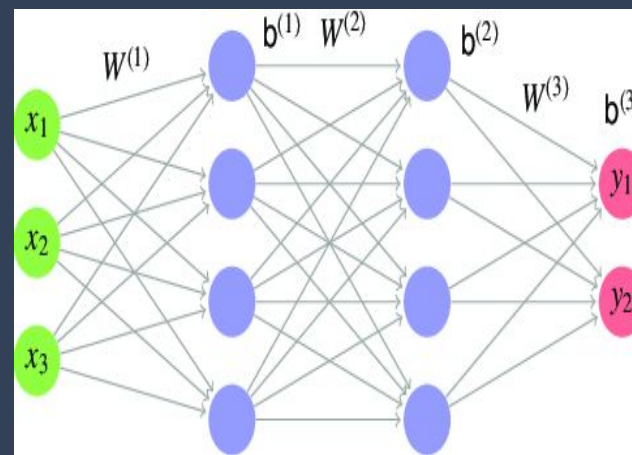
- It is feed forward
 - Connections from inputs towards outputs
 - No connection comes backwards
- It consists of layers
 - Current layer's input is previous layer's output
 - No lateral (intra-layer) connections
- That's it!

Types of FNN Models

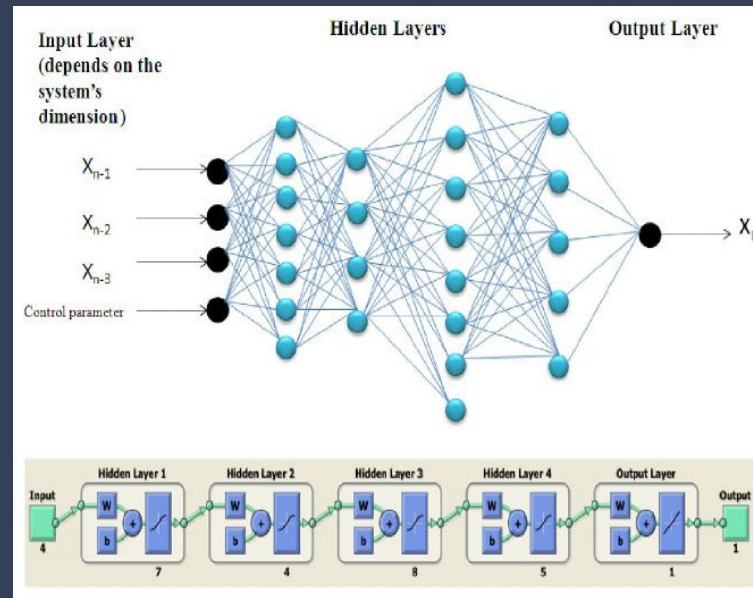
1. Single-Layer FNN



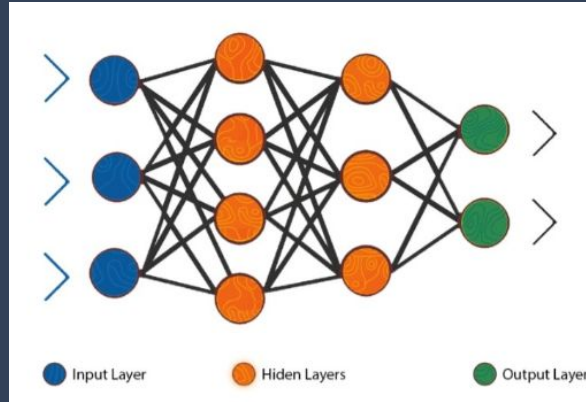
2. Multilayer FNN



3. Fully Connected FNN



Example



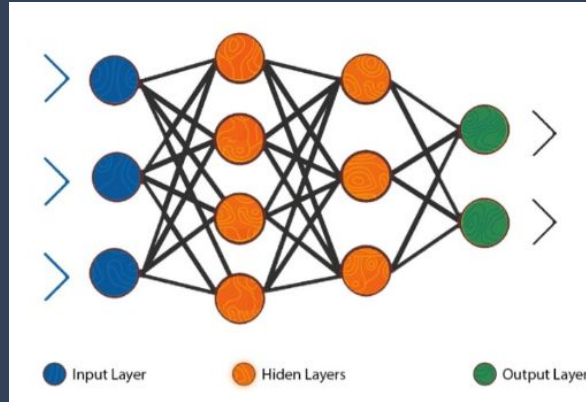
1. Trainable Parameters Between the Input layer and First Hidden Layer:

$$\text{weights} = 3 * 4 = 12$$

$$\text{biases} = 4 \text{ (4 nodes in 1st hidden layer)}$$

$$\text{Trainable Parameters} = \text{weights} + \text{biases} = 12 + 4 = 16$$

Example



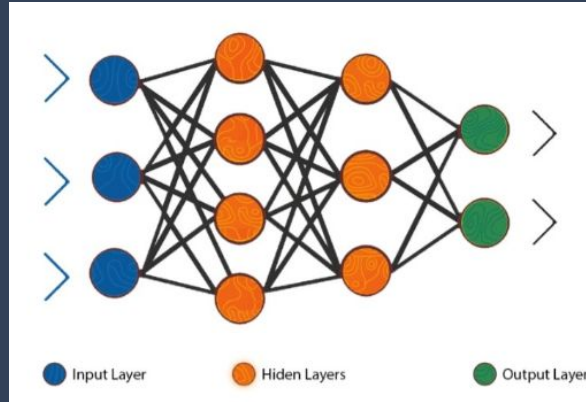
2. Trainable Parameters Between First Hidden layer and Second Hidden Layer:

weights = $4 * 3 = 12$

biases = 3 (3 nodes in 2nd hidden layer)

Trainable Parameters = weights + biases = $12 + 3 = 15$

Example



3. Trainable Parameters Between Second Hidden layer and Output Layer:

$$\text{weights} = 3 * 2 = 6$$

$$\text{biases} = 2 \text{ (2 nodes in output layer)}$$

$$\text{Trainable Parameters} = \text{weights} + \text{biases} = 6 + 2 = 8$$

Working example of FNN on dataset

[Neural Architectures](#) [Dataset 4 Live Class](#) [FNN](#) [Digit Recognizer Dataset.ipynb](#)

Advantages of FNN

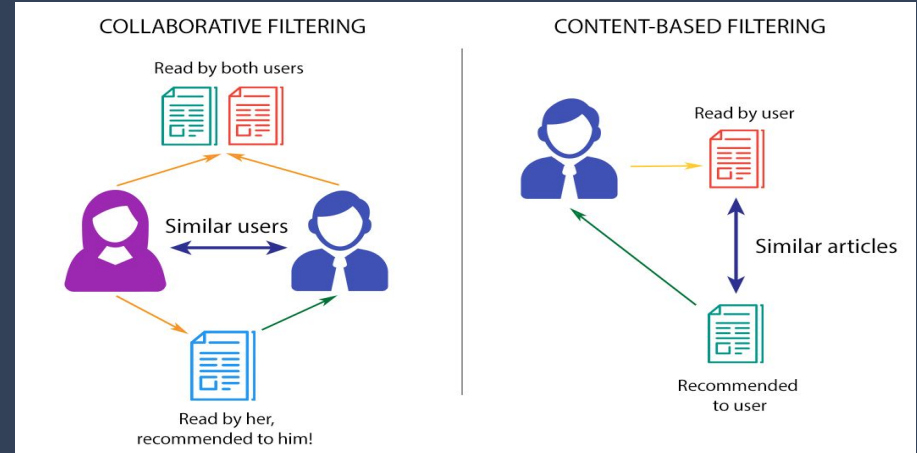
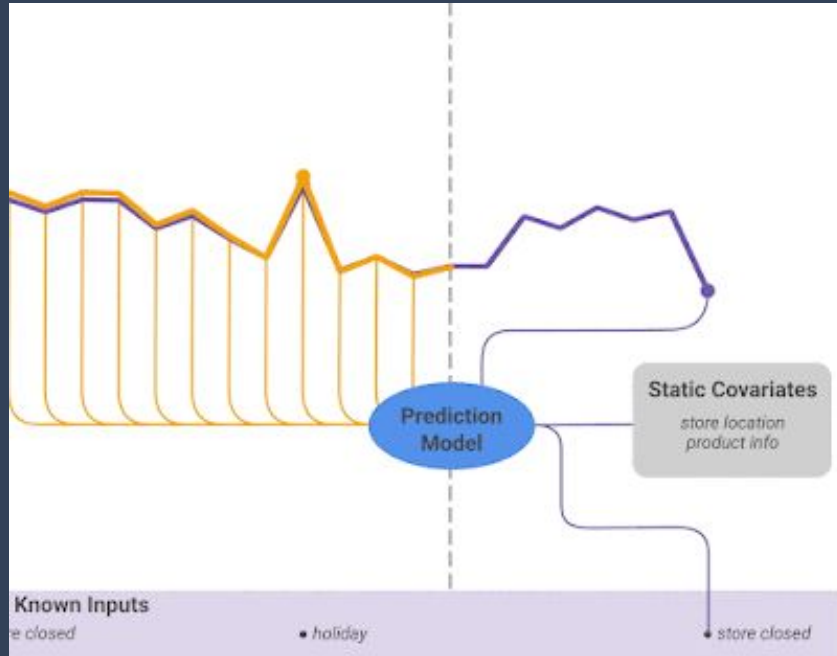
- Non-linear Modeling
- Universal Approximators
- Scalability and Parallelization
- End-to-End Learning
- Generalization
- Transfer Learning

Limitations of FNN

Challenges specific to FNNs when compared to other deep learning models

- Lack of Sequential Modeling
- Inefficient Parameter Sharing
- Handling Variable-Length Inputs
- Lack of Memory
- Interpretability

Applications of FNN



Section Summary

- ❏ What is Feed Forward Neural Network?
- ❏ Principle of Feed Forward Neural Network
- ❏ Discussion on Layers
- ❏ Types of Feed Forward Neural Network Models
- ❏ Advantages of FNN
- ❏ Limitations of FNN
- ❏ Applications of FNN
- ❏ Working example on dataset

Can you Answer these?

How Google photos tags the individuals in the photo?

OR

How does Self-driving Cars work?

OR

How does Snapchat filters are able to overlay mask, animation and digital effects on your face?

Convolutional Neural Networks (CNN)

Learning Objectives

Developing an understanding of CNN working, principles and its applications

Section Agenda

- ❑ Introduction to CNN
- ❑ Principle of CNN
- ❑ CNN Layers
- ❑ Types of FNN Models
- ❑ Advantages of CNN
- ❑ Limitations of CNN
- ❑ Applications of CNN
- ❑ Example of CNN on Dataset

How can FNN work on images?



Locality

Locality - An object's properties are primarily influenced by its immediate surroundings or nearby elements rather than distant ones

- **Nearby pixels are more strongly correlated**



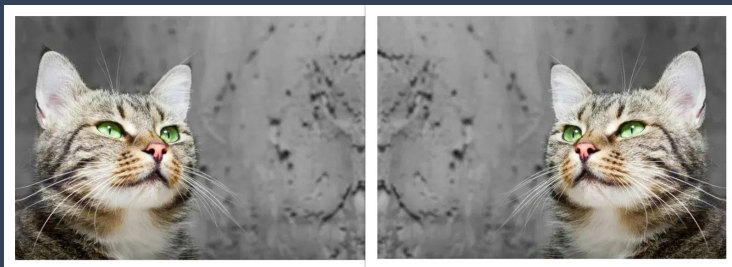
Nearby pixels are more strongly related than distant ones.

Objects are built up out of smaller parts.

Translation Invariance

Translational invariance - An object's properties remains unchanged or invariant when shifted or translated in space or time

- **Meaningful patterns can occur anywhere in the image**



Let's do some Math!

$$\begin{aligned} [\mathbf{H}]_{i,j} &= [\mathbf{U}]_{i,j} + \sum_k \sum_l [\mathbf{W}]_{i,j,k,l} [\mathbf{X}]_{k,l} \\ &= [\mathbf{U}]_{i,j} + \sum_a \sum_b [\mathbf{V}]_{i,j,a,b} [\mathbf{X}]_{i+a,j+b}. \end{aligned}$$

(a, b) only quantify the neighborhood from (i, j) and we taking contribution of intensities / X from those neighboring pixels weighted by V(a,b)

$$[\mathbf{V}]_{i,j,a,b} = [\mathbf{V}]_{a,b}$$

X - Input image
H - Hidden representation
U- Bias

By translation invariance

$$[\mathbf{H}]_{i,j} = u + \sum_a \sum_b [\mathbf{V}]_{a,b} [\mathbf{X}]_{i+a,j+b}.$$

Convolution!

Let's do some Math!

Locality -> We should not have to look very far away from location (i, j) in order to fetch relevant information to assess what is going on at $[H]_{i,j}$.

This means that outside some range $|a| > \Delta$ or $|b| > \Delta$, we should set $[V]_{a,b} = 0$.

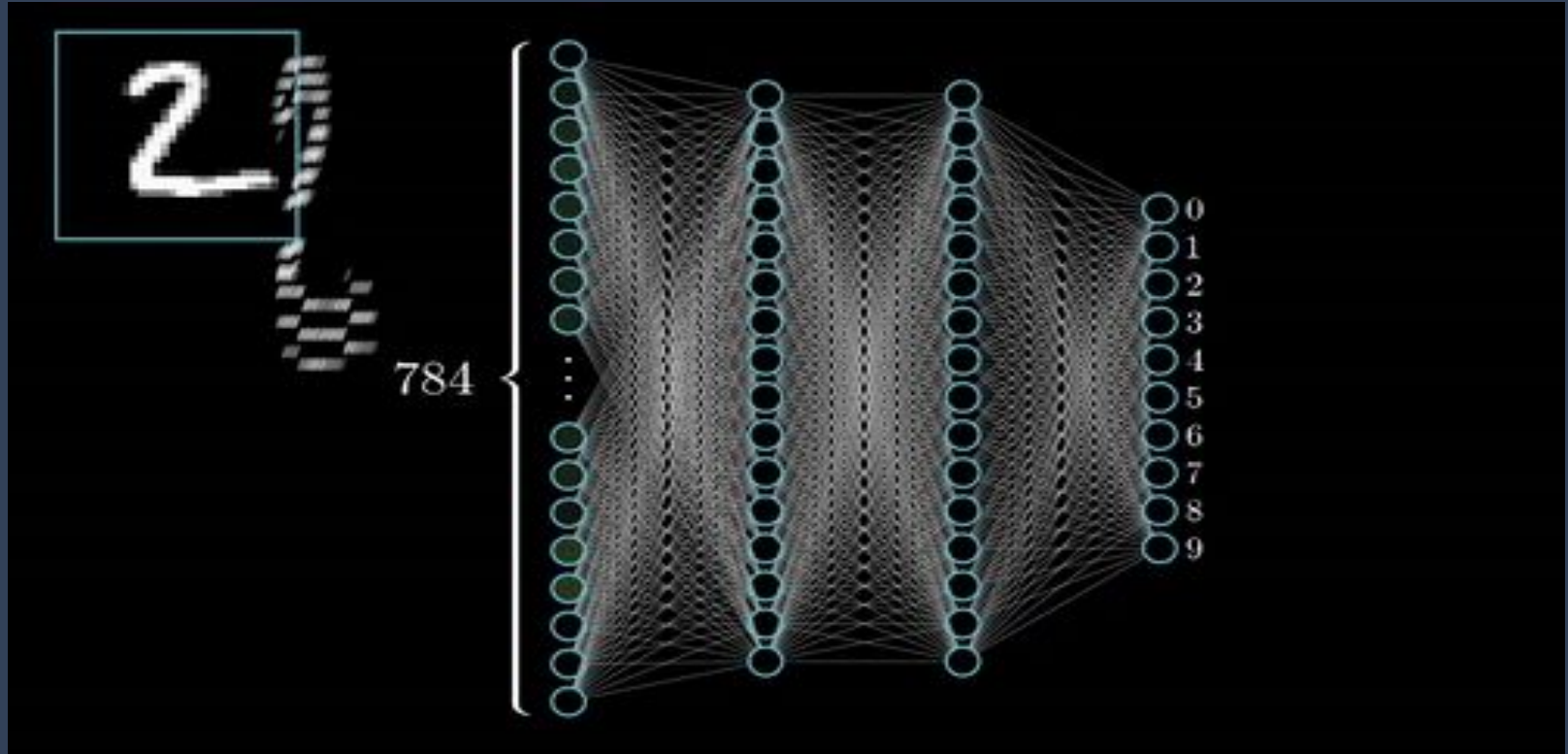
Alternatively, we can rewrite $[H]_{i,j}$ as

$$[\mathbf{H}]_{i,j} = u + \sum_{a=-\Delta}^{\Delta} \sum_{b=-\Delta}^{\Delta} [\mathbf{V}]_{a,b} [\mathbf{X}]_{i+a,j+b}.$$

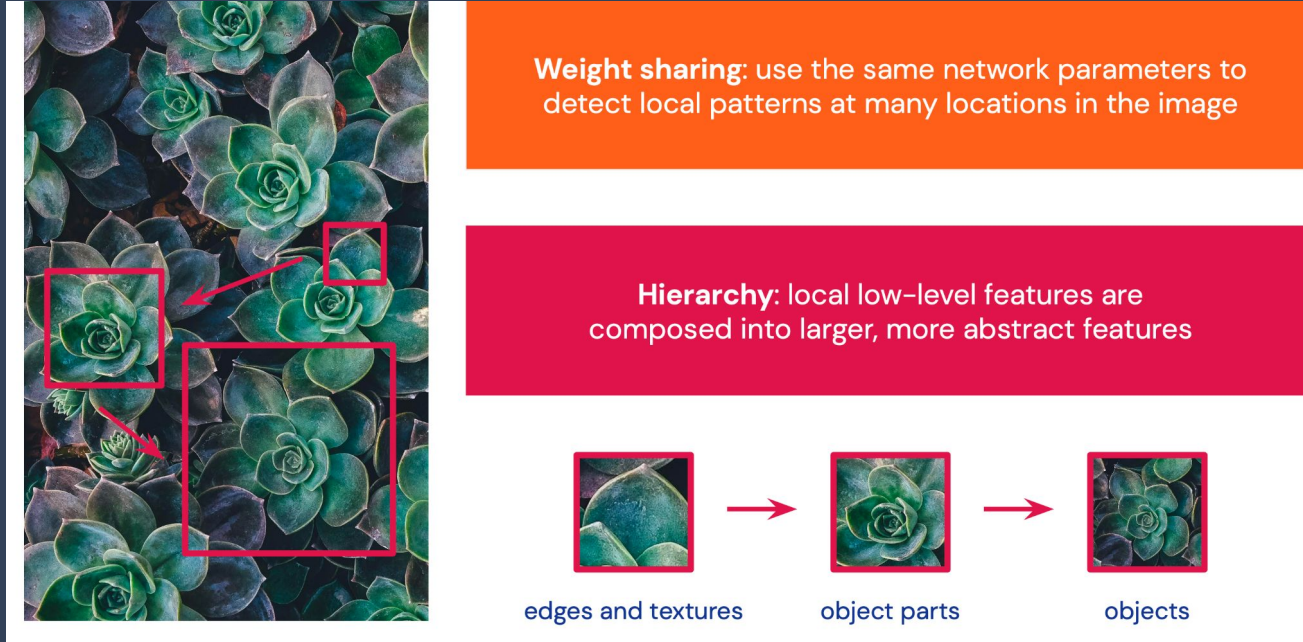
What is CNN?

- CNN stands for **C**onvolutional **N**eural **N**etwork. It is a type of deep neural network specifically designed for processing structured grid-like data, such as images or other forms of structured data like audio spectrograms or 2D sensor data.
- The key characteristic of CNN's is their ability to automatically learn and extract hierarchical representations from the input data. They achieve this through the use of
 - convolutional layers,
 - pooling layers, and
 - fully connected layers

CNN Data Flow



Principle of CNN



CNNs achieve this by employing convolutional layers, pooling layers, and fully connected layers.

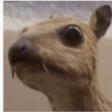
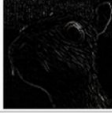
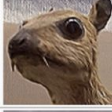
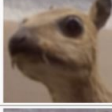
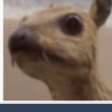
Convolutions

$$(f * g)(\mathbf{x}) = \int f(\mathbf{z})g(\mathbf{x} - \mathbf{z})d\mathbf{z}.$$

$$(f * g)(i, j) = \sum_a \sum_b f(a, b)g(i - a, j - b).$$

2D

Kernels - The Imperative!

Operation	Filter	Convolved Image
Identity	$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$	
Edge detection	$\begin{bmatrix} 1 & 0 & -1 \\ 0 & 0 & 0 \\ -1 & 0 & 1 \end{bmatrix}$	
	$\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$	
	$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$	
Sharpen	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	
Box blur (normalized)	$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$	
Gaussian blur (approximation)	$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$	

Why Kernels?

1. Local Feature Detection

- a. Small matrices (typically 3x3 or 5x5) that slide or convolve across the input image.
- b. Recognizes specific local patterns or features, such as edges, textures, corners, etc

1. Pattern Matching

- b. Weighted sum from convolution represents the presence of a particular feature that the kernel is designed to detect

1. Translation Invariance

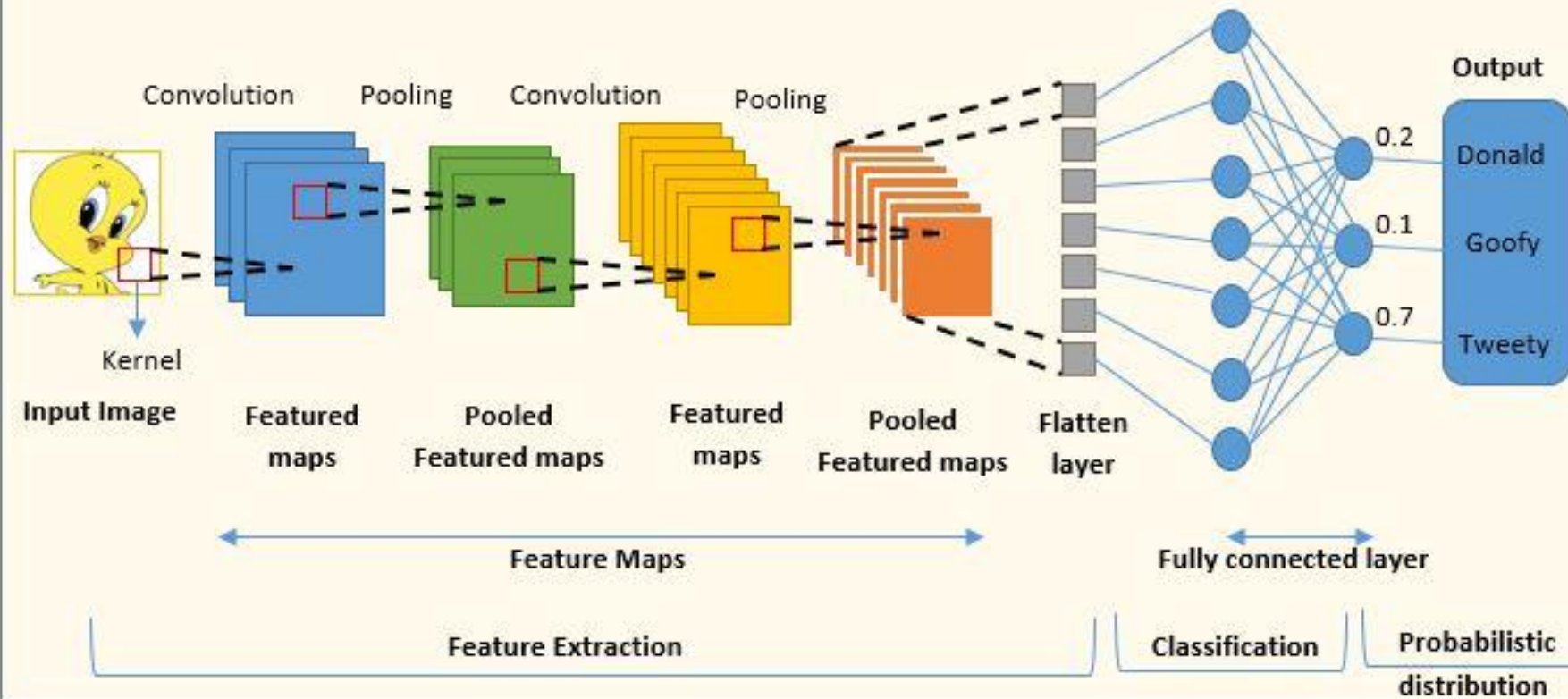
1. Hierarchy of Features

- b. CNNs use multiple kernels to detect features at different scales and complexities.
- c. In deeper layers, kernels combine information from lower-level features to detect more abstract and complex patterns.

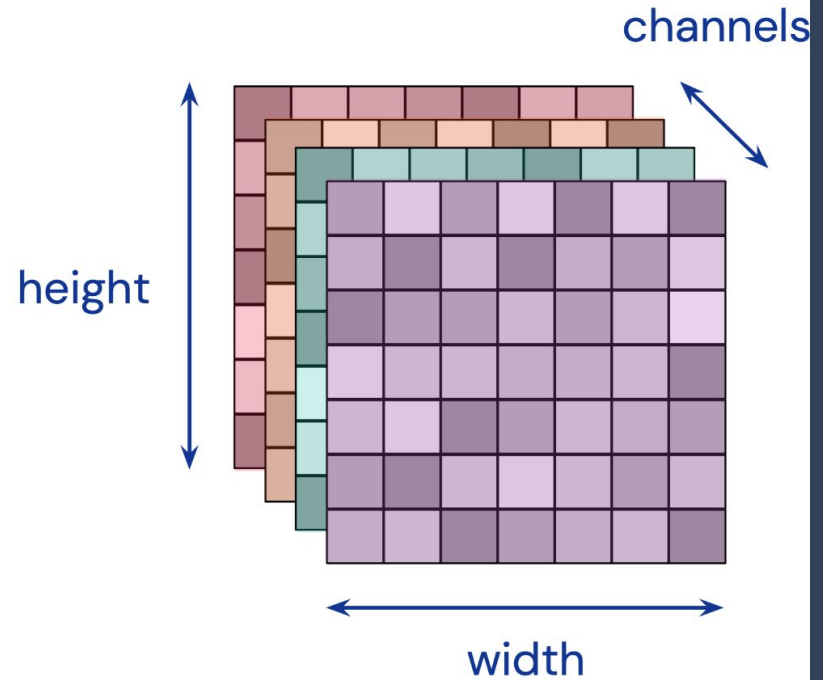
Layers of CNN

1. **Input Layer:** It receives the raw input data, such as an image or a sequence of audio samples.
2. **Convolutional Layers:** They are the core building blocks of CNNs.
3. **Activation Function:** It is applied element-wise to the output of each convolutional layer.
4. **Pooling Layers:** They reduce the spatial dimensions of the feature maps by downsampling
5. **Fully Connected Layers:** Also known as dense layers, they are used at the end of the CNN architecture to perform high-level reasoning and classification
6. **Output Layer:** It produces the final predictions or outputs of the network

A Typical Convolutional Neural Network (CNN)



Channels



Components of CNN

❑ Convolution

Convolution involves applying a set of learnable filters (kernels) to input data, sliding them over the input spatially, and performing element-wise multiplication and summation to extract local features.

Convolutions help capture spatial relationships and detect patterns.

Components of CNN

1. Input image $\rightarrow W_1 \times H_1 \times D_1$
2. Requires four hyperparameters:
 - a. Number of filters K
 - b. their spatial extent F
 - c. the stride S
 - d. the amount of zero padding P
3. Output
 - a. $W_2 = (W_1 - F + 2P) / S + 1$
 - b. $H_2 = (H_1 - F + 2P) / S + 1$
 - c. $D_2 = K$

Example: 2D Convolution

1. $2 \times 3 \times 5$ Flipped Kernel: $0.5 \times 1 + 0.5 \times 1 \times 0.5 + 2 \times 2 + 3 \times 0.5$

The diagram illustrates a 2D convolution operation. On the left is a 5x5 input matrix with values:

7	2	3	3	8
4	5	3	8	4
3	3	2	8	4
2	8	7	2	7
5	4	4	5	4

. In the center is a 3x3 kernel matrix with values:

1	0	-1
1	0	-1
1	0	-1

. To the right of the kernel is an equals sign followed by a 3x3 output matrix. The top-left cell of the output matrix contains the value 6. Below the kernel matrix, the calculation for the value 6 is shown: $7 \times 1 + 4 \times 1 + 3 \times 1 + 2 \times 0 + 5 \times 0 + 3 \times 0 + 3 \times -1 + 3 \times -1 + 2 \times -1 = 6$.

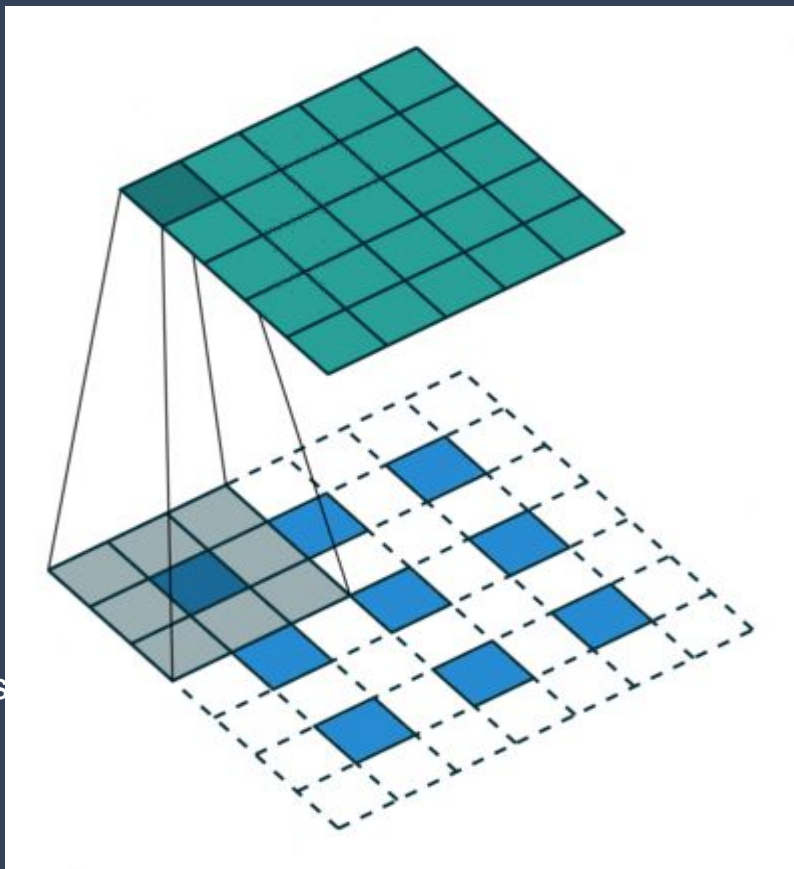
❏ Stride

Stride is a parameter in convolutional operations that determines the step size at which the convolutional filters (kernels) move across the input data.

It controls the spatial displacement of the filters, impacting the output size and computational complexity.

Example: 2D Convolution with Stride

Edit: <https://hannibunny.github.io/mlbook/neuralnetworks>

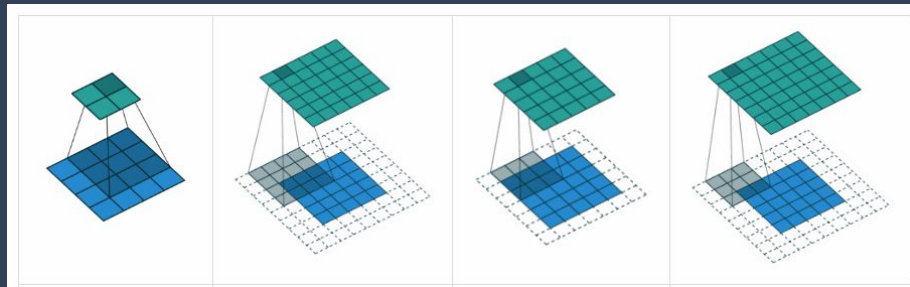


❏ Padding

Padding is a technique used in CNNs to add extra border pixels to the input data before applying convolutions. Padding helps preserve spatial dimensions and avoid information loss at the edges of the input.

If you have an input size of W (width) by H (height), using a filter of size F by F , a stride of S , and padding of P , the output size can be determined with the following formula:

- Width of Output: $(W - F + 2P) / S + 1$
- Height of Output: $(H - F + 2P) / S + 1$



Input		Kernel		Output																																													
<table><tr><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>0</td><td>1</td><td>2</td><td>0</td></tr><tr><td>0</td><td>3</td><td>4</td><td>5</td><td>0</td></tr><tr><td>0</td><td>6</td><td>7</td><td>8</td><td>0</td></tr><tr><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td></tr></table>	0	0	0	0	0	0	0	1	2	0	0	3	4	5	0	0	6	7	8	0	0	0	0	0	0	*	<table><tr><td>0</td><td>1</td></tr><tr><td>2</td><td>3</td></tr></table>	0	1	2	3	=	<table><tr><td>0</td><td>3</td><td>8</td><td>4</td></tr><tr><td>9</td><td>19</td><td>25</td><td>10</td></tr><tr><td>21</td><td>37</td><td>43</td><td>16</td></tr><tr><td>6</td><td>7</td><td>8</td><td>0</td></tr></table>	0	3	8	4	9	19	25	10	21	37	43	16	6	7	8	0
0	0	0	0	0																																													
0	0	1	2	0																																													
0	3	4	5	0																																													
0	6	7	8	0																																													
0	0	0	0	0																																													
0	1																																																
2	3																																																
0	3	8	4																																														
9	19	25	10																																														
21	37	43	16																																														
6	7	8	0																																														

❏ Pooling

Pooling is a downsampling operation commonly used in CNNs to reduce the spatial dimensions of feature maps while retaining important information.

Pooling helps to make the learned features more robust to small spatial variations and reduces computational complexity.

There are different types of pooling, with max pooling being the most widely used.

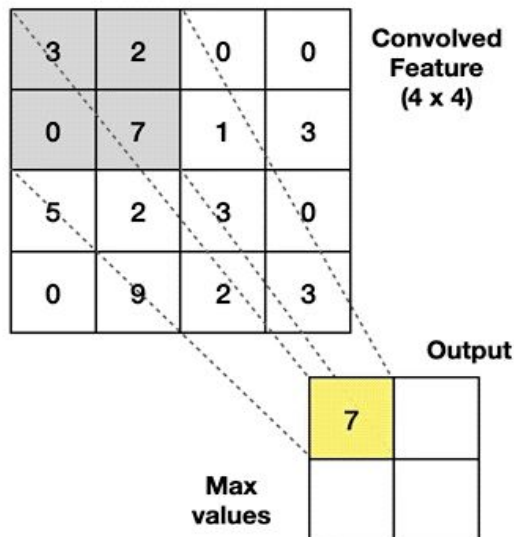
- Accepts a volume of size $W_1 \times H_1 \times D_1$
- Requires two hyperparameters:
 - their spatial extent F ,
 - the stride S ,
- Produces a volume of size $W_2 \times H_2 \times D_2$ where:
 - $W_2 = (W_1 - F)/S + 1$
 - $H_2 = (H_1 - F)/S + 1$
 - $D_2 = D_1$

❏ Pooling

Max Pooling

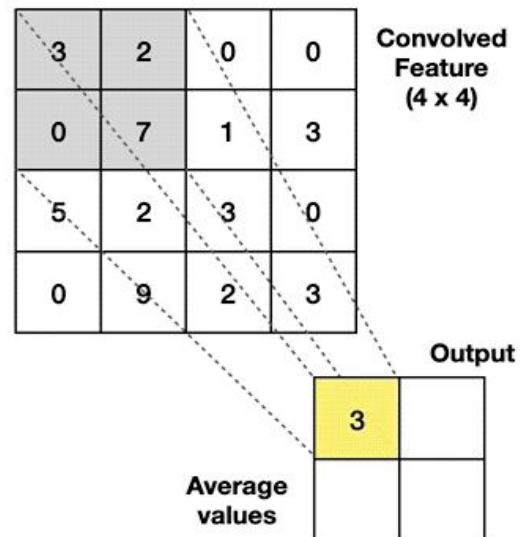
Take the **highest** value from the area covered by the kernel

Example: Kernel of size 2 x 2; stride=(2,2)



Average Pooling

Calculate the **average** value from the area covered by the kernel



What is the purpose of flattening in stages

Flattening is a common operation used in the intermediate stages of a neural network, typically in the transition from convolutional layers to fully connected layers.

The purpose of flattening is to reshape the multidimensional feature maps generated by convolutional and pooling layers into a one-dimensional vector. Here are the primary purposes of flattening in neural networks:

- Transition to Fully Connected Layers
- Vectorized Input
- Parameterization
- Scalability and Flexibility

Prevention of overfitting

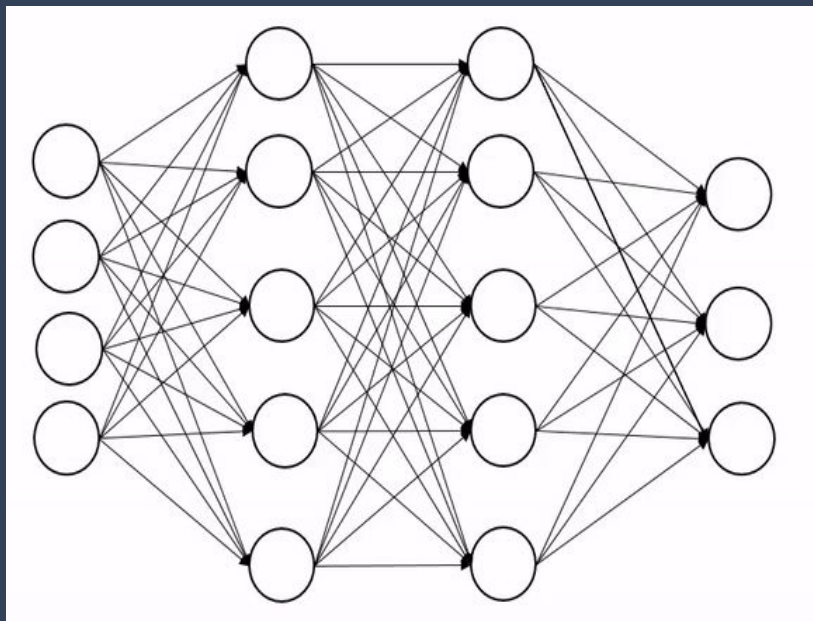
1. Dropout
2. L1/L2 regularization
3. Batch normalization
4. Early stopping

Dropout

Dropout is a regularization technique commonly used in neural networks, including Convolutional Neural Networks (CNNs), to prevent overfitting.

It involves randomly setting a fraction of the neuron outputs to zero during the training process.

This technique helps improve the generalization ability of the network by preventing complex co-adaptations between neurons and encouraging robust feature learning.



L1/L2 Regularization

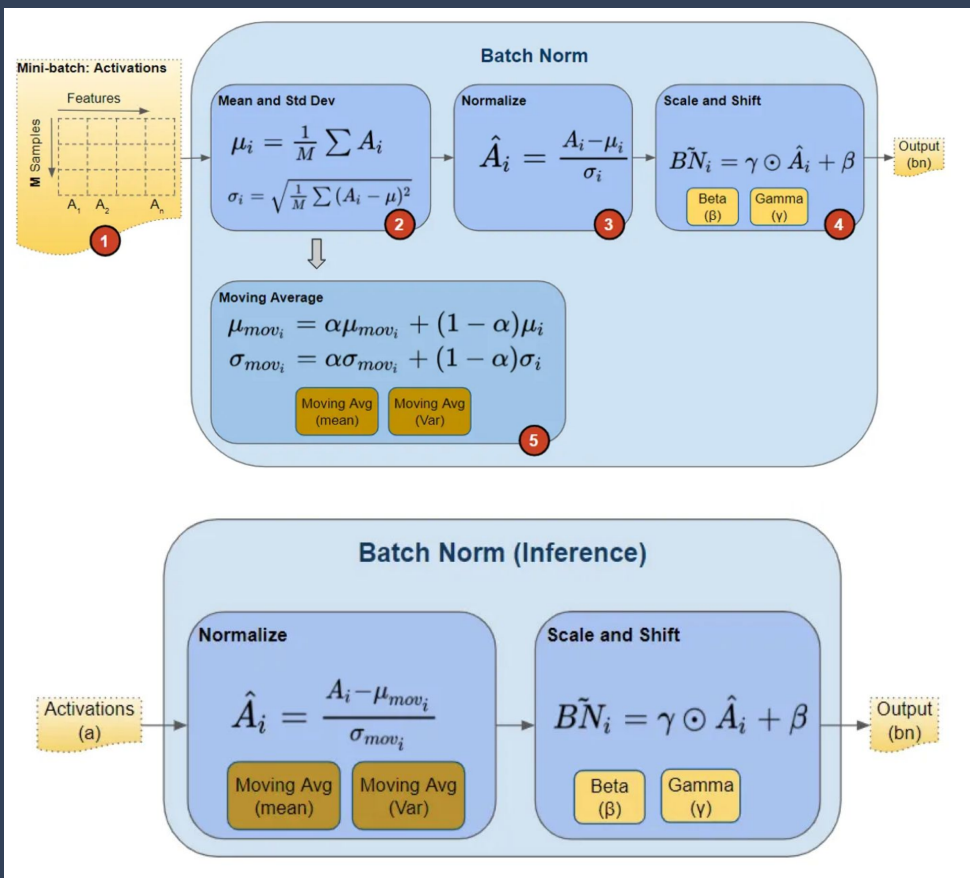
$$LossFunction = \frac{1}{N} \sum_{i=1}^N (\hat{Y} - Y)^2 + \lambda \sum_{i=1}^N |\theta_i|$$

L1

$$LossFunction = \frac{1}{N} \sum_{i=1}^N (\hat{Y} - Y)^2 + \lambda \sum_{i=1}^N \theta_i^2$$

L2

Batch Normalization

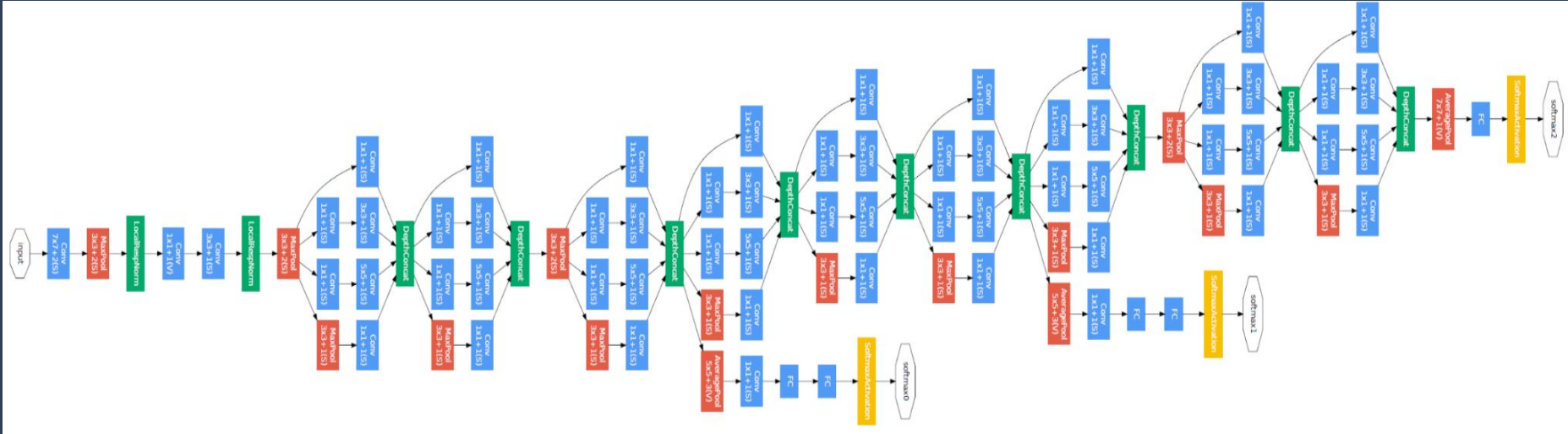


Types of CNN Models

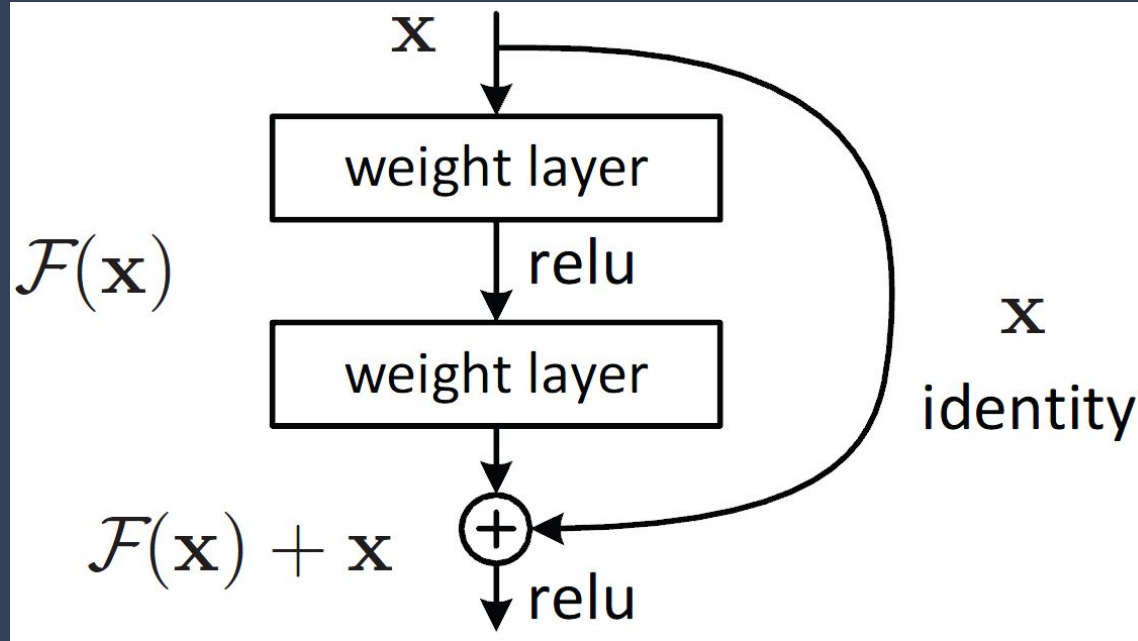
There are several types of Convolutional Neural Network (CNN) models, each with its own architecture and characteristics. Here are some commonly used types of CNN models:

- 1) LeNet-5
- 2) AlexNet
- 3) VGGNet
- 4) GoogLeNet (Inception)
- 5) ResNet
- 6) DenseNet
- 7) MobileNet
- 8) EfficientNet
- 9) U-Net

GoogLeNet Architecture

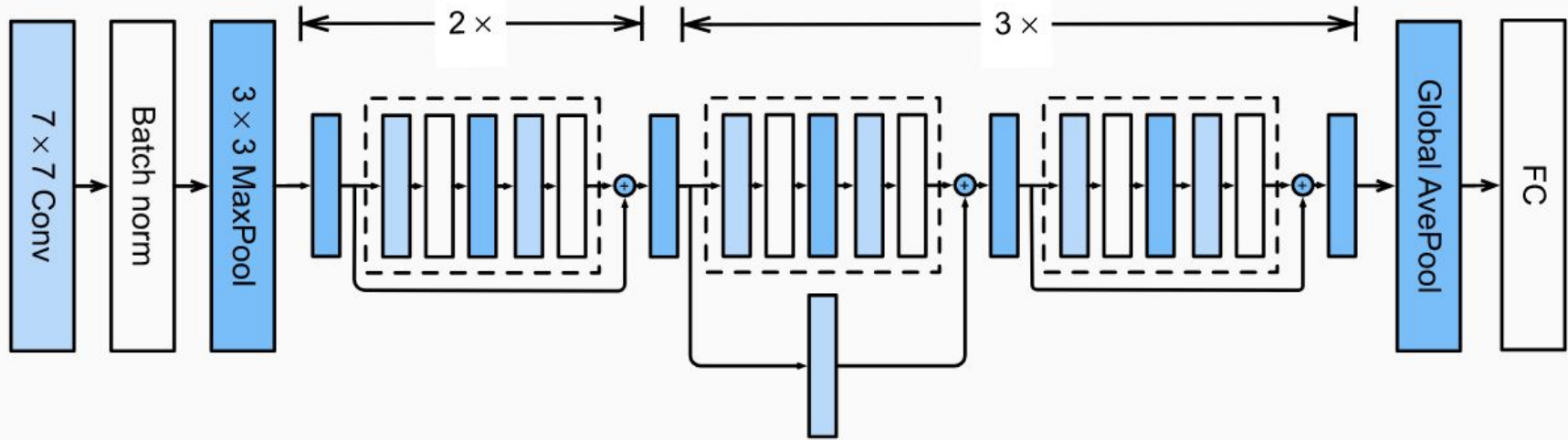


ResNet Architecture



Residual connections facilitate training deeper networks

ResNet Architecture



ResNet Vs VGG VS Inception Vs Transformer (Additional Read)

Architecture	ResNet (Residual Network)	VGG (Visual Geometry Group)	Inception (GoogLeNet)	Transformer
Architecture Style	Introduced the concept of residual blocks, which contain skip connections or shortcuts. These shortcuts allow the network to learn residual functions, making it easier to train very deep networks without vanishing gradients.	Consist of a series of convolutional layers followed by max-pooling layers. The network architecture is characterized by its simplicity, using 3x3 convolutional filters with a fixed stride and padding.	Uses inception modules, which are made up of multiple parallel convolutional layers with different filter sizes. This allows the network to capture features at different scales.	Architecture was originally designed for NLP tasks but has since been adapted for computer vision. It uses self-attention mechanisms to capture relationships between input elements. These consist of encoder and decoder layers, and they operate on sequences of data.
Depth	Can be very deep, with hundreds of layers. They are known for their depth.	Not as deep as ResNets. VGG16 and VGG19 are common variants.	Relatively deep but are not as deep as ResNets. The original GoogLeNet had 22 layers.	Depth is often limited by computational constraints.
Use Case	Used for tasks like image classification, object detection, and segmentation, where very deep networks are required.	Known for its strong feature extraction capabilities and is often used for image classification and feature extraction.	Designed for efficient use of computational resources and have been used in image classification and object detection tasks.	Highly versatile and have been used for a wide range of tasks, including machine translation, image captioning, image generation, and even reinforcement learning. ⁵

Advantages of CNN

Convolutional Neural Networks (CNNs) offer several advantages for various computer vision tasks.

- Local Connectivity and Parameter Sharing
- Translation Invariance
- Reduced Parameter Count
- Hierarchical Feature Learning
- Effective Use of Convolutional and Pooling Layers
- GPU Acceleration

Limitations with CNN

- Lack of Long-Term Memory
- Difficulty in Handling Irregular Input Shapes
- Computational and Memory Requirements
- Data Augmentation and Large-Scale Datasets
- Interpretability and Explainability

Real-life application of CNN

Circulation revenue has increased by 5% in Finland. // Positive

Panostaja did not disclose the purchase price. // Neutral

Paying off the national debt will be extremely painful. // Negative

The company anticipated its operating profit to improve. // _____

LM

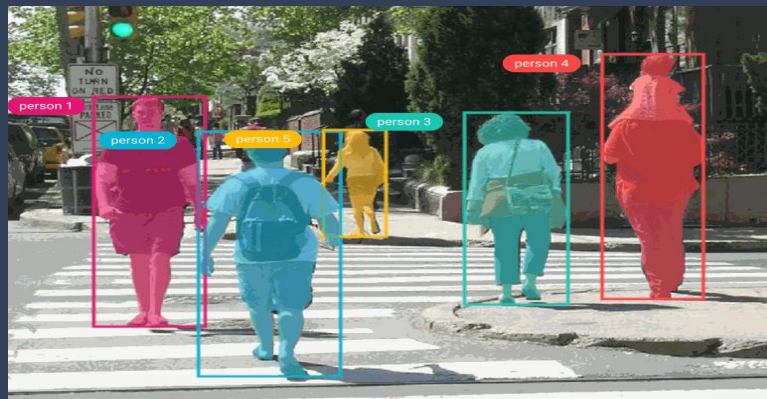
Circulation revenue has increased by 5% in Finland. // Finance

They defeated ... in the NFC Championship Game. // Sports

Apple ... development of in-house chips. // Tech

The company anticipated its operating profit to improve. // _____

LM



Milan is a city in northern Italy, capital of Lombardy, and the second-most populous city in Italy after Rome. Milan served as the capital of the Western Roman Empire, the Duchy of Milan and the Kingdom of Lombardy-Venetia. The city proper has a population of about 1.4 million while its metropolitan city has 3.26 million inhabitants. Its continuously built-up urban area, that stretches well beyond the boundaries of the administrative metropolitan city, is the fourth largest in the EU with 5.27 million inhabitants.

Working example of CNN on dataset

[Neural Architectures _ Dataset 1 - Live Class _ CNN _ Digit Recognizer Dataset.ipynb](#)

https://colab.research.google.com/github/robertovlp/Neural-Networks-Challenge/blob/master/Neural_Networks_Challenge.ipynb

Section Summary

- ❏ What is Convolutional Neural Network?
- ❏ Principle of Convolutional Neural Network
- ❏ Discussion on Layers and Components
- ❏ Types of Convolutional Neural Network Models
- ❏ Advantages of CNN
- ❏ Limitations of CNN
- ❏ Applications of CNN
- ❏ Working example on dataset

Can you Answer these?

How do Amazon's forecasting system work?

OR

How does Amazon echo and apple Siri understand the speech?

OR

How does gmail frames automatic replies for your emails?

Recurrent Neural Networks (RNN)

Learning Objectives

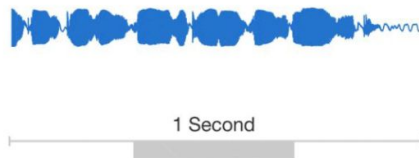
Developing an understanding of RNN working, principle and its applications

Section Agenda

- ❑ Introduction to RNN
- ❑ Principle of RNN
- ❑ RNN Layers
- ❑ Types of RNN Models
- ❑ Advantages of RNN
- ❑ Limitations of RNN
- ❑ Applications of RNN
- ❑ Example of RNN on Dataset

What are Sequences?

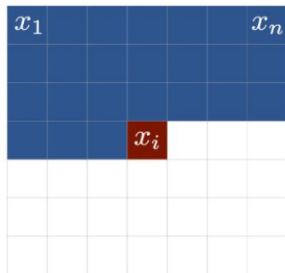
"Sequences really seem to be everywhere! We should learn how to model them. What is the best way to do that? Stay tuned!"



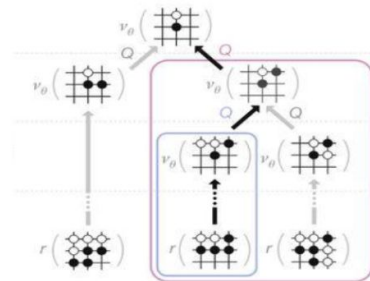
Words, letters

Speech

Videos



```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def forward_backward_prop(w, T):
5     hs = [0.5]
6     for t in range(T):
7         hs.append(np.tanh(w*hs[-1]))
8
9     dh = 1
10    for t in range(T):
11        dh = (1-hs[-1:t]**2) * w * dh
12
13    return hs[-1], dh
14
15 T = 10 # sequence length
16 wlim = 4 # limit of interval over weights w
17
18 results = []
19 ws = np.linspace(-wlim, wlim, 1000)
20 for w in ws:
21     results.append(forward_backward_prop(w, T))
22
23 plt.plot(ws, [r[0] for r in results], label='RNN state')
24 plt.plot(ws, [r[1] for r in results], label='Gradients')
```



Images

Programs

Decision making

We will try to understand this from a text point of view

Modeling word probabilities is really difficult

Simplest model:

Assume independence of words

$$p(\mathbf{x}) = \prod_{t=1}^T p(x_t)$$

$$p(\text{"modeling"}) \times p(\text{"word"}) \times p(\text{"probabilities"}) \times p(\text{"is"}) \times p(\text{"really"}) \times p(\text{"difficult"})$$



Modeling $p(\mathbf{x})$

The chain rule

Computing the joint $p(\mathbf{x})$ from conditionals

$$p(\mathbf{x}) = \prod_{t=1}^T p(x_t | x_1, \dots, x_{t-1})$$

Modeling

Modeling **word**

Modeling word **probabilities**

Modeling word probabilities **is**

Modeling word probabilities is **really**

Modeling word probabilities is really **difficult**

$$p(x_1)$$

$$p(x_2 | x_1)$$

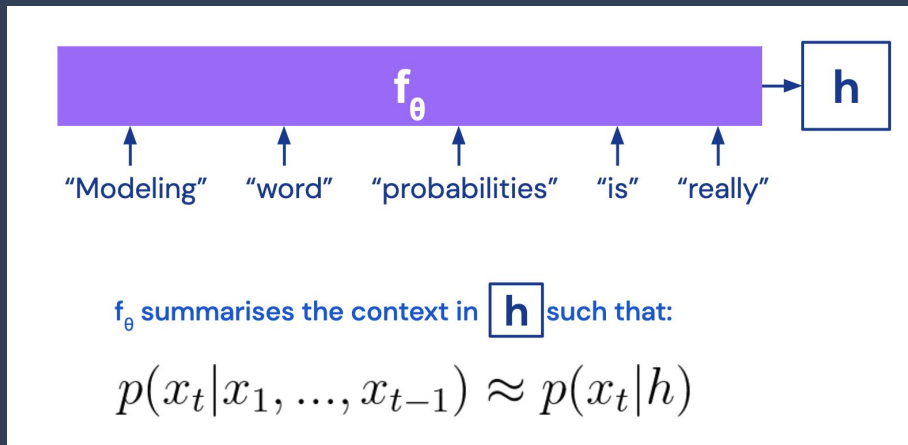
$$p(x_3 | x_2, x_1)$$

$$p(x_4 | x_3, x_2, x_1)$$

$$p(x_5 | x_4, x_3, x_2, x_1)$$

$$p(x_6 | x_5, x_4, x_3, x_2, x_1)$$

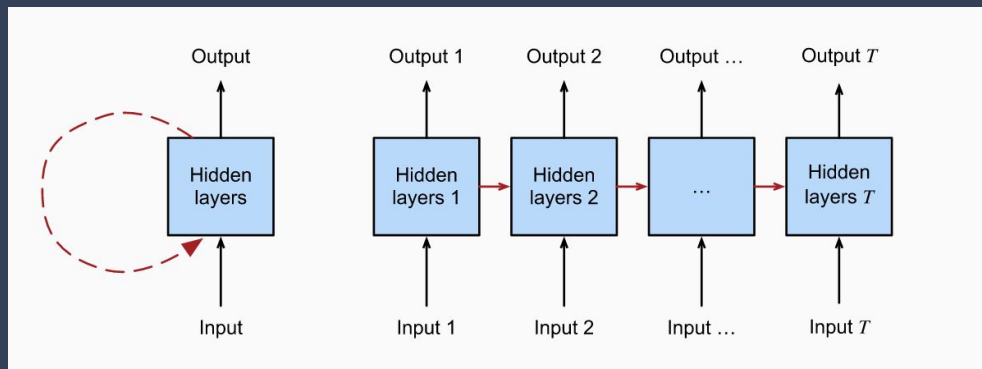
What is the right way to learn word probabilities then?



Vectorize the context such that for f_θ

- Order matters
- Variable length
- Learnable (differentiable)
- Individual changes can have large effects(non-linear/deep)

Recurrent Neural Networks (RNN)!



- RNN stands for **R**ecurrent **N**eural **N**etwork. It is a type of artificial neural network designed to process sequential data, such as time series, speech, natural language, and handwriting.
- RNNs have a recurrent connection that allows them to maintain and utilize information from previous steps or time points.
- RNNs maintain an internal state, often referred to as a "hidden state" or "memory,"

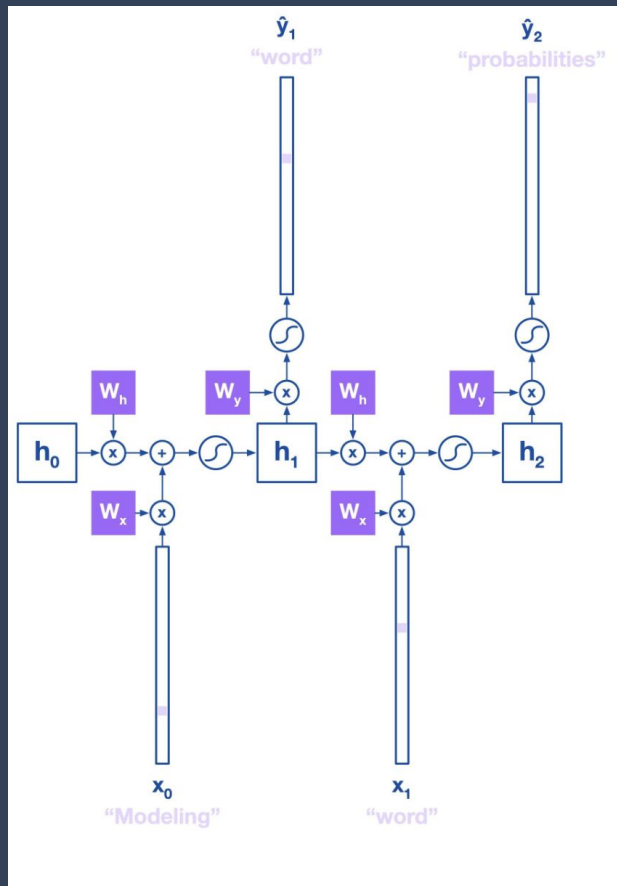
Principle of RNN

- The principle of RNNs is to process sequential data by maintaining and utilizing information from previous time steps.
- RNNs achieve this by introducing recurrent connections, which allow the network to retain and update hidden states or memories as it processes the sequential input.

The main principle of RNN can be summarized as follows:

- Sequential Processing
- Hidden State Update
- Information Flow and Feedback
- Weight Sharing
- Training with Backpropagation through Time

Recurrent Neural Networks (RNN)



$$p(\mathbf{y}_{t+1}) = \text{softmax}(\mathbf{W}_y \mathbf{h}_t)$$

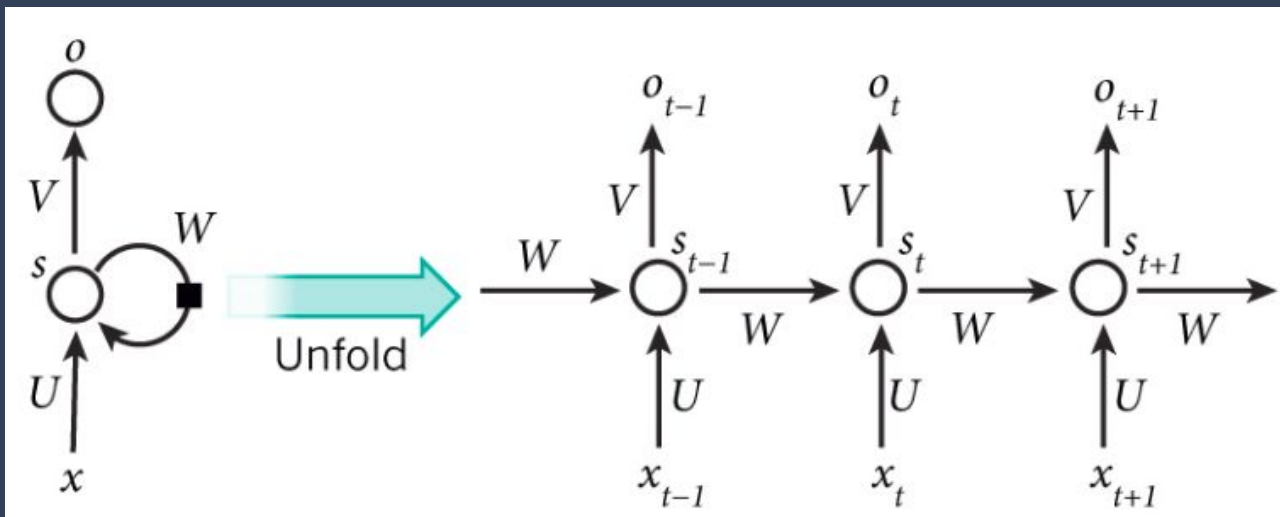
$$\mathbf{h}_t = \tanh(\mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{W}_x \mathbf{x}_t)$$

Cross Entropy Loss for full sentence

$$\mathcal{L}_{\theta}(\mathbf{y}, \hat{\mathbf{y}}) = - \sum_{t=1}^T \mathbf{y}_t \log \hat{\mathbf{y}}_t$$

RNN - Backward propagation through time (BPTT)

BPTT works by unrolling all input timesteps. Each timestep has one input timestep, one copy of the network, and one output. Errors are then calculated and accumulated for each timestep.



- U are the weights associated with the inputs x_t
- W are the weights associated with the hidden state of the RNN
- V are the weights associated with the outputs of the RNN

RNN - Backward propagation through time (BPTT)

$$\begin{cases} h_t = Ux_t + Wh_{t-1} \\ o_t = Vh_t \end{cases}$$

where :

h_t : is the hidden state

o_t : is the output at time step t

x_t : is the input at time step t

$$L = \frac{1}{T} \sum_{t=1}^T l(o_t, y_t)$$

$$\begin{cases} \frac{\partial L}{\partial V} = \frac{\partial L}{\partial o_t} \cdot \frac{\partial o_t}{\partial V} \\ \frac{\partial L}{\partial o_t} = \frac{\partial l(o_t, y_t)}{T \cdot \partial o_t} \\ \frac{\partial o_t}{\partial V} = h_t \\ \frac{\partial L}{\partial V} = \sum_{t=1}^T \frac{\partial l}{\partial o_t} \cdot h_t^T \end{cases}$$

first derivative that we need to compute is the derivative with respect to the weights $\mathbf{V}$ that are associated with the outputs $\mathbf{o}_t$

$$\frac{\partial L_{t+1}}{\partial W} = \frac{\partial L_{t+1}}{\partial o_{t+1}} \cdot \frac{\partial o_{t+1}}{\partial h_{t+1}} \cdot \frac{\partial h_{t+1}}{\partial h_t} \cdot \frac{\partial h_t}{\partial W}$$

the next gradient we should compute is the one with respect to $\mathbf{W}$. We know that the hidden state at time step $t+1$ is dependent on the hidden state at time step t

RNN - Backward propagation through time (BPTT)

$$\frac{\partial L_{t+1}}{\partial W} = \sum_{k=1}^{t+1} \frac{\partial L_{t+1}}{\partial o_{t+1}} \cdot \frac{\partial o_{t+1}}{\partial h_{t+1}} \cdot \frac{\partial h_{t+1}}{\partial h_k} \cdot \frac{\partial h_k}{\partial W}$$

From multivariable chain rule

$$\frac{\partial L}{\partial W} = \sum_t^T \sum_{k=1}^{t+1} \frac{\partial L_{t+1}}{\partial o_{t+1}} \cdot \frac{\partial o_{t+1}}{\partial h_{t+1}} \cdot \frac{\partial h_{t+1}}{\partial h_k} \cdot \frac{\partial h_k}{\partial W}$$

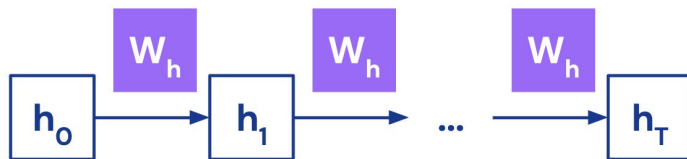
$$\frac{\partial L}{\partial U} = \sum_t^T \sum_{k=1}^{t+1} \frac{\partial L_{t+1}}{\partial o_{t+1}} \cdot \frac{\partial o_{t+1}}{\partial h_{t+1}} \cdot \frac{\partial h_{t+1}}{\partial h_k} \cdot \frac{\partial h_k}{\partial U}$$

Gradient with respect to **U**

$$\begin{cases} \frac{\partial L}{\partial V} = \sum_{t=1}^T \frac{\partial l}{\partial o_t} \cdot h_t^\top \\ \frac{\partial L}{\partial W} = \sum_t^T \sum_{k=1}^{t+1} \frac{\partial L_{t+1}}{\partial o_{t+1}} \cdot \frac{\partial o_{t+1}}{\partial h_{t+1}} \cdot \frac{\partial h_{t+1}}{\partial h_k} \cdot \frac{\partial h_k}{\partial W} \\ \frac{\partial L}{\partial U} = \sum_t^T \sum_{k=1}^{t+1} \frac{\partial L_{t+1}}{\partial o_{t+1}} \cdot \frac{\partial o_{t+1}}{\partial h_{t+1}} \cdot \frac{\partial h_{t+1}}{\partial h_k} \cdot \frac{\partial h_k}{\partial U} \end{cases}$$

Gradient of the hidden state at time step **t+1** is dependent on the hidden state at time step **k** !

Problem of Vanishing Gradients



$$\mathbf{h}_t = \mathbf{W}_h \mathbf{h}_{t-1} \qquad \mathbf{h}_t \rightarrow \infty \text{ if } |\mathbf{W}_h| > 1$$

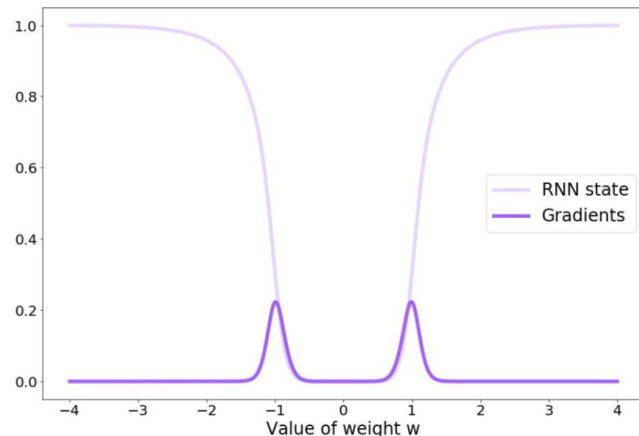
$$\mathbf{h}_t = (\mathbf{W}_h)^t \mathbf{h}_0 \qquad \mathbf{h}_t \rightarrow 0 \text{ if } |\mathbf{W}_h| < 1$$

$$\mathbf{h}_t = \mathbf{W}_h \mathbf{h}_{t-1} \rightarrow \mathbf{h}_t = \tanh(\mathbf{W}_h \mathbf{h}_{t-1})$$

Problem of Vanishing Gradients

$$\mathbf{h}_t = \tanh(\mathbf{W}_h \mathbf{h}_{t-1})$$

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def forward_backward_prop(w, T):
5     hs = [0.5]
6     for _ in range(T):
7         hs.append(np.tanh(w*hs[-1]))
8
9     dh = 1
10    for t in range(T):
11        dh = (1-hs[-1-t]**2) * w * dh
12
13    return hs[-1], dh
14
15 T = 10 # sequence length
16 wlim = 4 # limit of interval over weights w
17
18 results = []
19 ws = np.linspace(-wlim, wlim, 1000)
20 for w in ws:
21     results.append(forward_backward_prop(w, T))
22
23 plt.plot(ws, [r[0] for r in results], label='RNN state')
24 plt.plot(ws, [r[1] for r in results], label='Gradients')
```



$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-T}} = (1 - \tanh^2(\mathbf{W}_h \mathbf{h}_{t-1})) \mathbf{W}_h \frac{\partial \mathbf{h}_{t-1}}{\partial \mathbf{h}_{t-T}}$$

Vanishing/Exploding Gradients

- Vanishing gradients occur when the gradients diminish exponentially as they are back propagated through multiple time steps. As a result, the RNN has difficulty learning long-term dependencies and capturing relevant information from distant time steps.
 - This problem is especially prominent in traditional RNN architectures, such as the simple RNN, due to the nature of the activation function and the repeated multiplication of weights.
- On the other hand, exploding gradients occur when the gradients grow exponentially during backpropagation. This leads to large updates to the network's parameters, which can cause instability and prevent the network from converging to a good solution.

Types of RNN Models

Some commonly used types of RNN models:

- 1) Vanilla RNN
- 2) Long Short-Term Memory (LSTM)
- 3) Gated Recurrent Unit (GRU)
- 4) Bidirectional RNN (BiRNN)
- 5) Hierarchical RNN
- 6) Attention-based RNN

Advantages of RNN

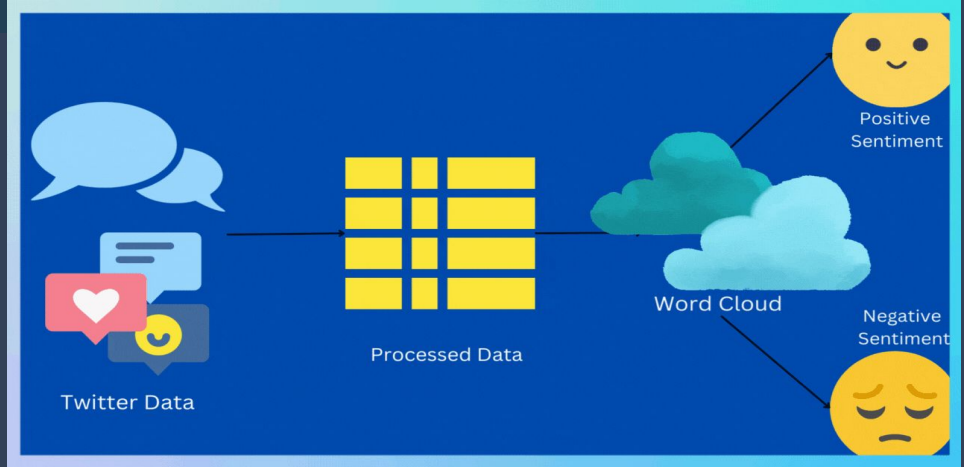
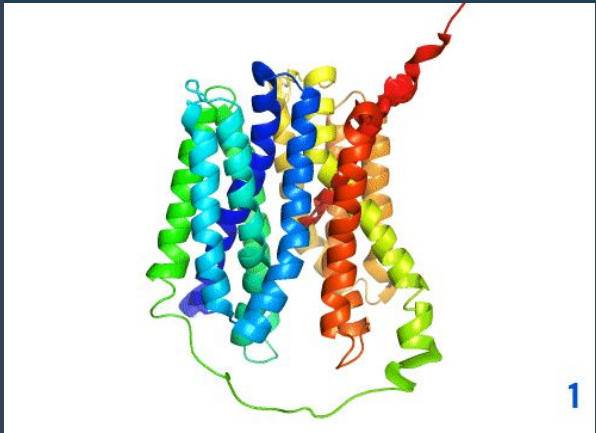
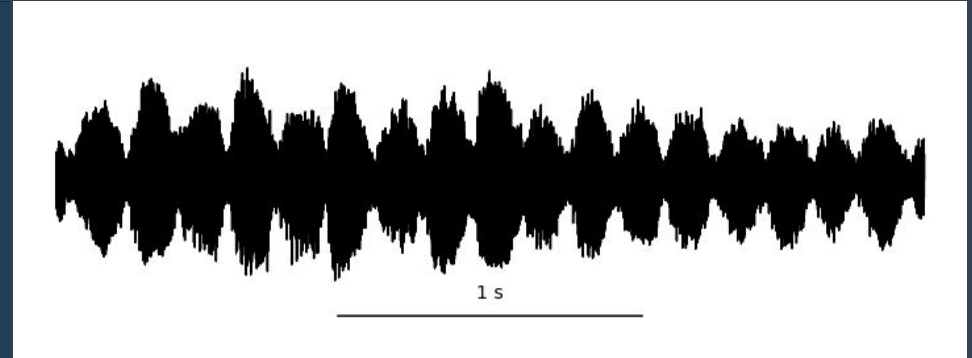
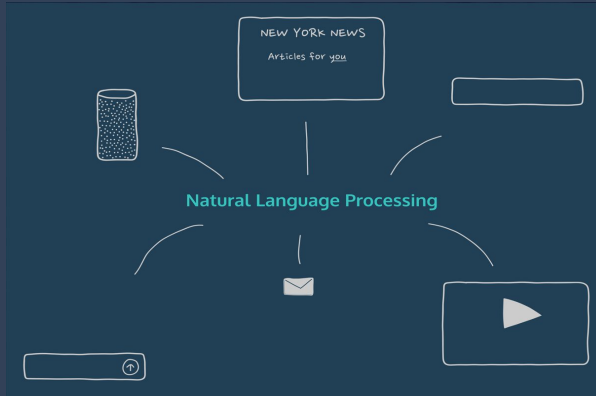
Recurrent Neural Networks (RNNs) offer several advantages that make them suitable for various tasks involving sequential data. Here are some key advantages of RNNs:

- Sequential Modeling
- Memory and Context Preservation
- Variable-Length Input Handling
- Parameter Sharing
- Time-Series Analysis
- Language Modeling and Generation
- Online and Streaming Data

Limitations of RNN

- Difficulty Capturing very Long-Term Dependencies
- Lack of Explicit Memory
- Sequential Computation Limitations
- Difficulty with Variable-Length Inputs
- Gradient Vanishing and Exploding
- Lack of Parallelization
- Difficulty in Handling Irregular Time Steps

Real-life application of RNN



Working example of RNN on dataset

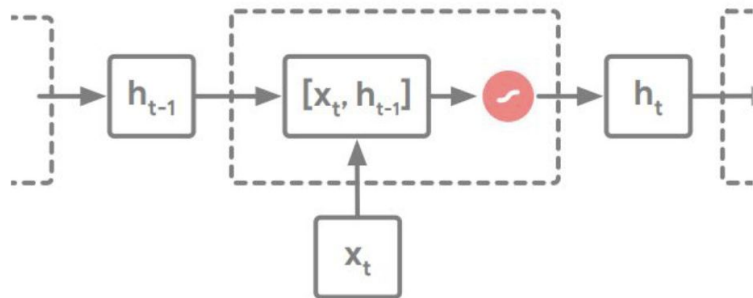
[Neural Architectures _ Dataset 2- Live Class _ RNN-LSTM _ New York Stock Exchange Dataset.ipynb](#)

Long Short-Term Memory (LSTM) networks

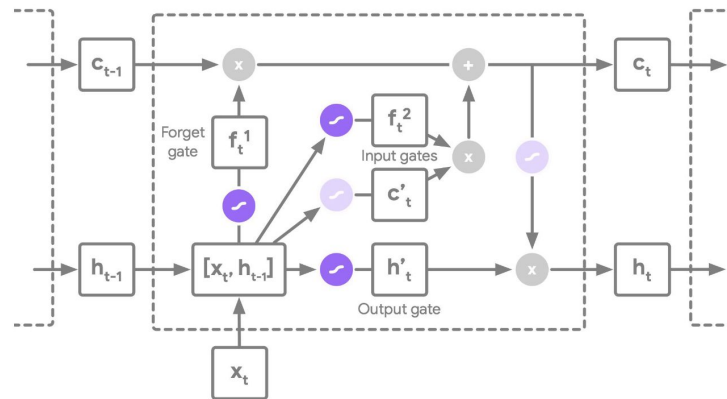
Long Short-Term Memory (LSTM) architecture consists of several layers that work together to capture long-term dependencies in sequential data.

LSTMs are explicitly designed to avoid the long-term dependency problem.

RNN state update



LSTM state update

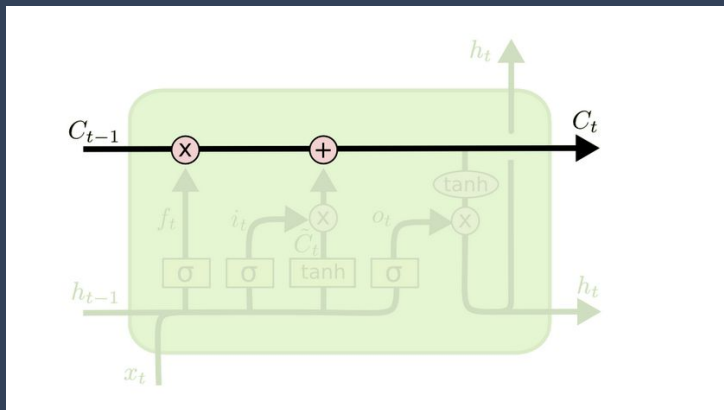


Core components of LSTM

- Cell State
- Forget Gate
- Input Gates
- Output Gate

Don't worry, we will look into each one of them step by step

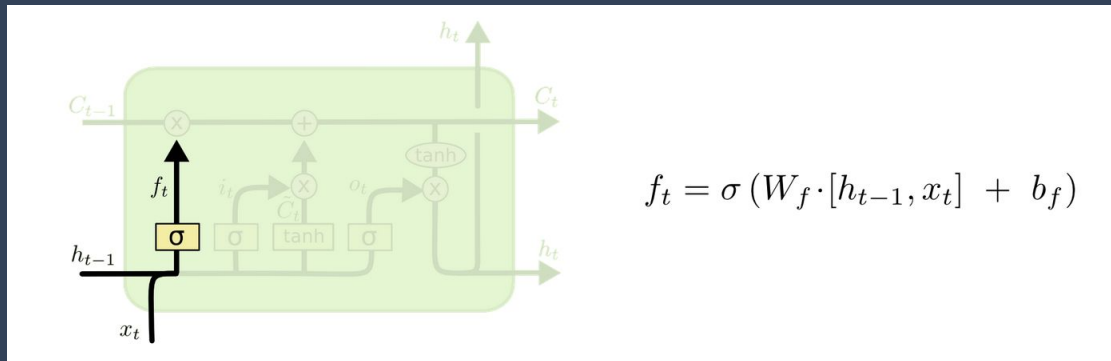
LSTM - Cell State



The cell state is kind of like a conveyor belt of information.

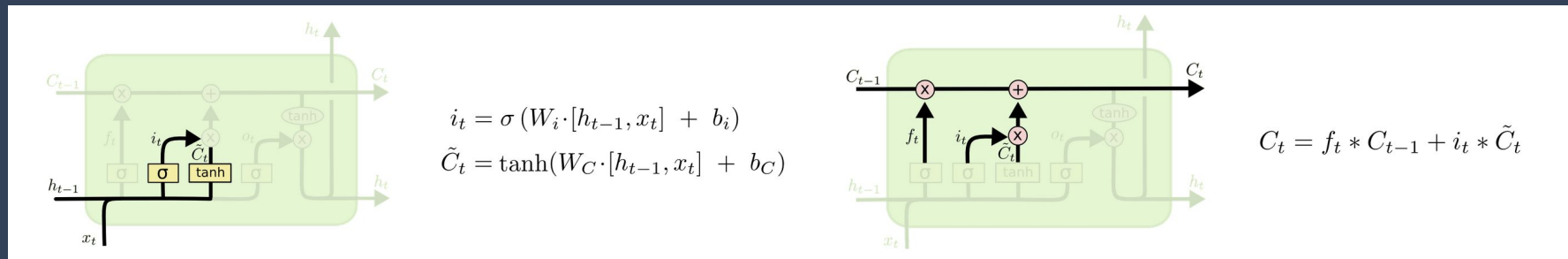
- The LSTM does have the ability to remove or add information to the cell state, regulated by various gates.

LSTM - Forget Gate



Forget Gate decided what information we're going to **retain** and what we are going to **filter out** from the cell state using *Sigmoid*

LSTM - Input Gate

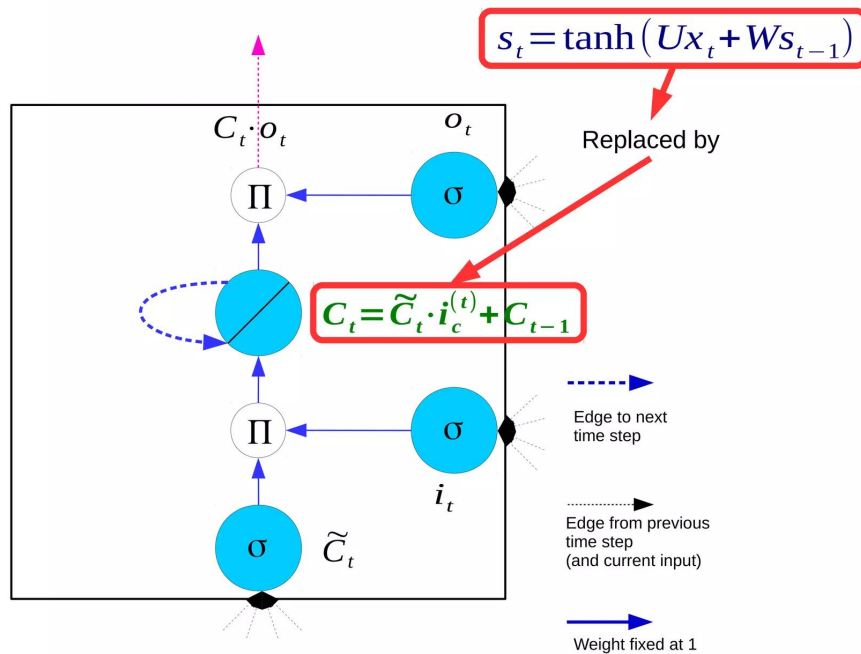


Input Gate decides what new information we're going to store in the cell state. This has three parts.

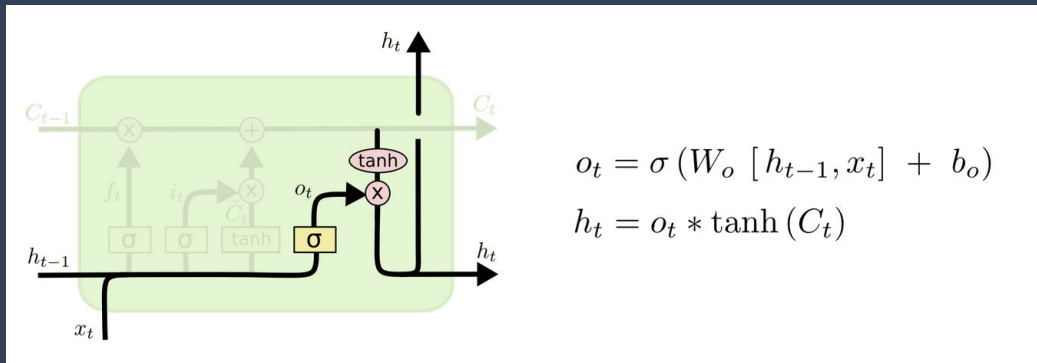
- a *sigmoid* layer called the “*input gate layer*” decides which values we'll update
- a *tanh* layer creates a vector of new candidate values, $\tilde{C}_t$, that could be added to the state
- combine b & c to create an update to the state

Vanishing Gradient - Forget Gate and Constant Error Carousel (CEC)

Constant error carousel



LSTM - Output Gate



Output gate decides what we're going to emit at the end. This will be based on our cell state, but will be a filtered version.

- sigmoid layer decides what parts of the cell state we're going to output
- cell state is put through *tanh* and multiplied by the output of the sigmoid gate to get only the the parts we decided to output

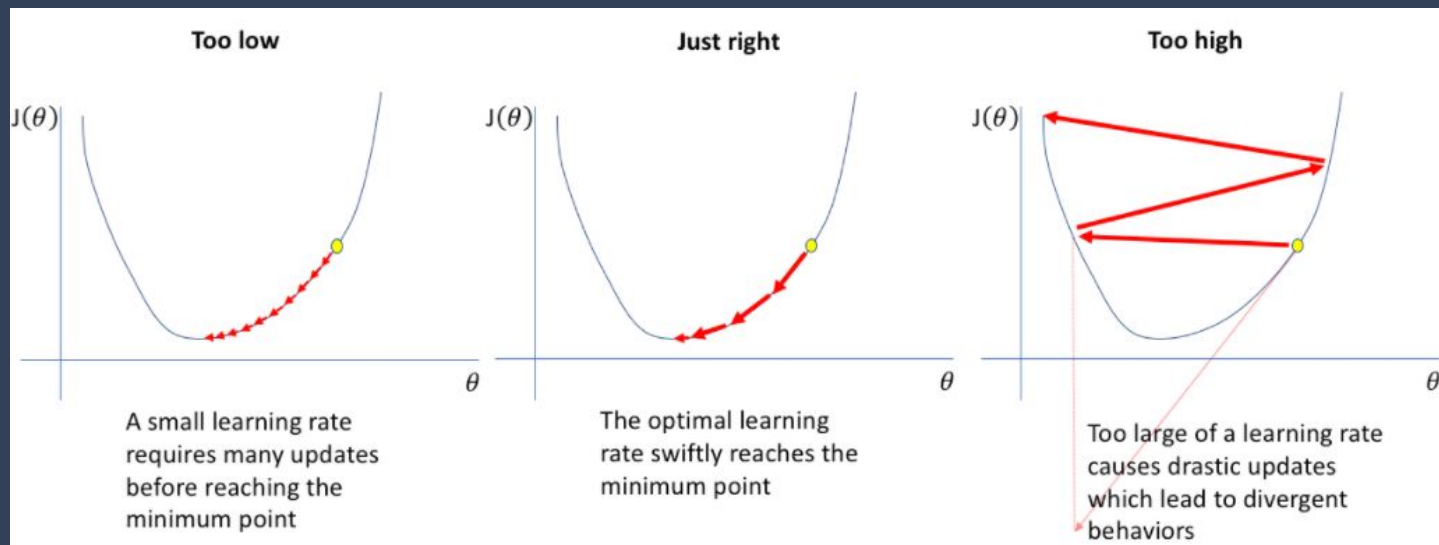
Section Summary

- What is Recurrent Neural Network?
- Principle of Recurrent Neural Network
- Discussion on Layers and Components
- Types of Recurrent Neural Network Models
- Deep dive into LSTMs
- Advantages of RNN
- Limitations of RNN
- Applications of RNN
- Working example on dataset

Miscellaneous Topics

- Effect of Learning Rate
- Loss Functions
- Overfitting and Underfitting
- Batch Size
- Weight Sharing in CNN and RNN

Effect of learning rate



The learning rate controls how big of a step for an optimizer to reach the minima of the loss function

- Large learning rate - algorithm learns fast but may oscillate around or even jump over the minima
- Small learning rate - very slow convergence

Learning rate schedule as a hyperparameter

Learning rate schedule is a predefined framework that adjusts the learning rate between epochs or iterations as the training progresses. Two of the most common techniques for learning rate schedule are:

- Constant learning rate: we initialize a learning rate and don't change it during training
- Learning rate decay: we select an initial learning rate, then gradually reduce it in accordance with a scheduler

```
def lr_time_based_decay(epoch, lr):  
    return lr * 1 / (1 + decay * epoch)
```

where `decay` is a parameter that is normally calculated as:

```
decay = initial_learning_rate/epochs
```

Loss Function in Deep Learning

- Mean Squared Loss - typical used for regression tasks

$$E(\mathbf{w}) = \frac{1}{n} \sum_i^n (y_i - f(x_i))^2$$

- Cross Entropy Loss - typical used for classification tasks

- Binary Cross Entropy - for binary classification
- Categorical Cross Entropy - for multi-class classification when the target labels are one-hot encoded

$$L_{BCE} = -\frac{1}{n} \sum_{i=1}^n (Y_i \cdot \log \hat{Y}_i + (1 - Y_i) \cdot \log (1 - \hat{Y}_i))$$

$$E = -\frac{1}{N} \sum_{n=1}^N \sum_{k=1}^K y_{nk} \ln t_{nk} + \alpha \|\mathbf{w}\|_2^2$$

- Contrastive Loss - It takes a pair of inputs and minimizes the embedding distance when they are from the same class but maximizes the distance otherwise
 - It is typically used in multimodal learning, such as in the [CLIP](#) (Contrastive Language-Image Pretraining) model

$$\ell_{i,j} = -\log \frac{\exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_j)/\tau)}{\sum_{k=1}^{2N} \sum_{[k \neq i]} \exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_k)/\tau)},$$

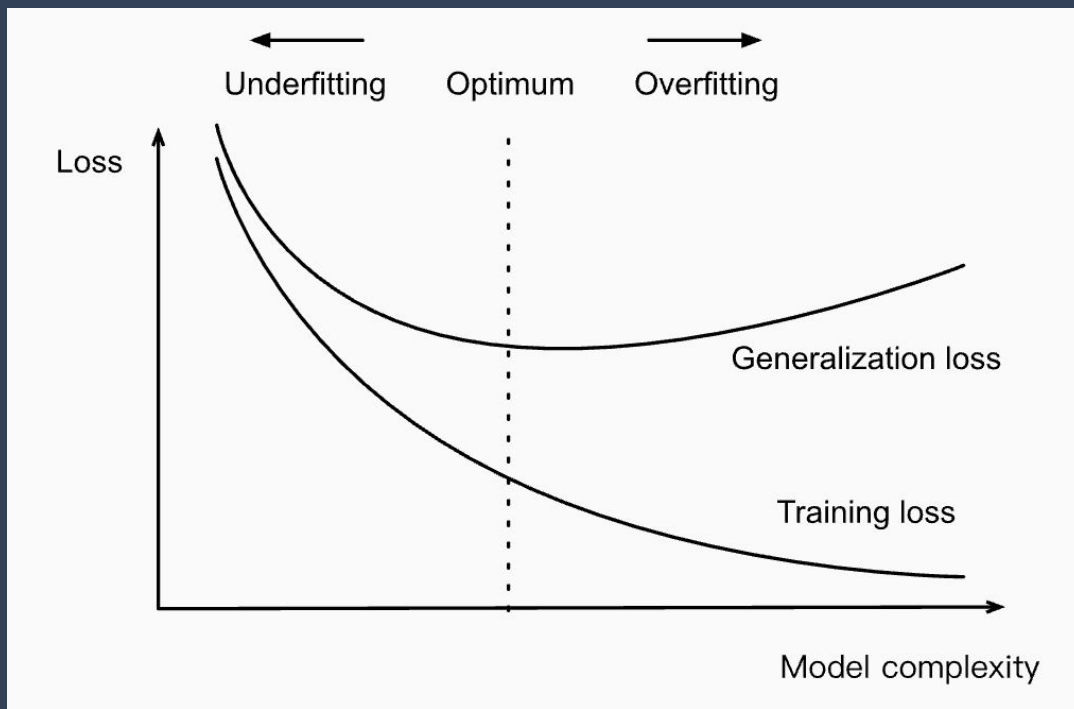
Why is loss monitored vs the metric itself

- Differentiability
- Gradient Descent
- Direct Optimization
- Early Stopping
- Interpretability
- Computational Efficiency

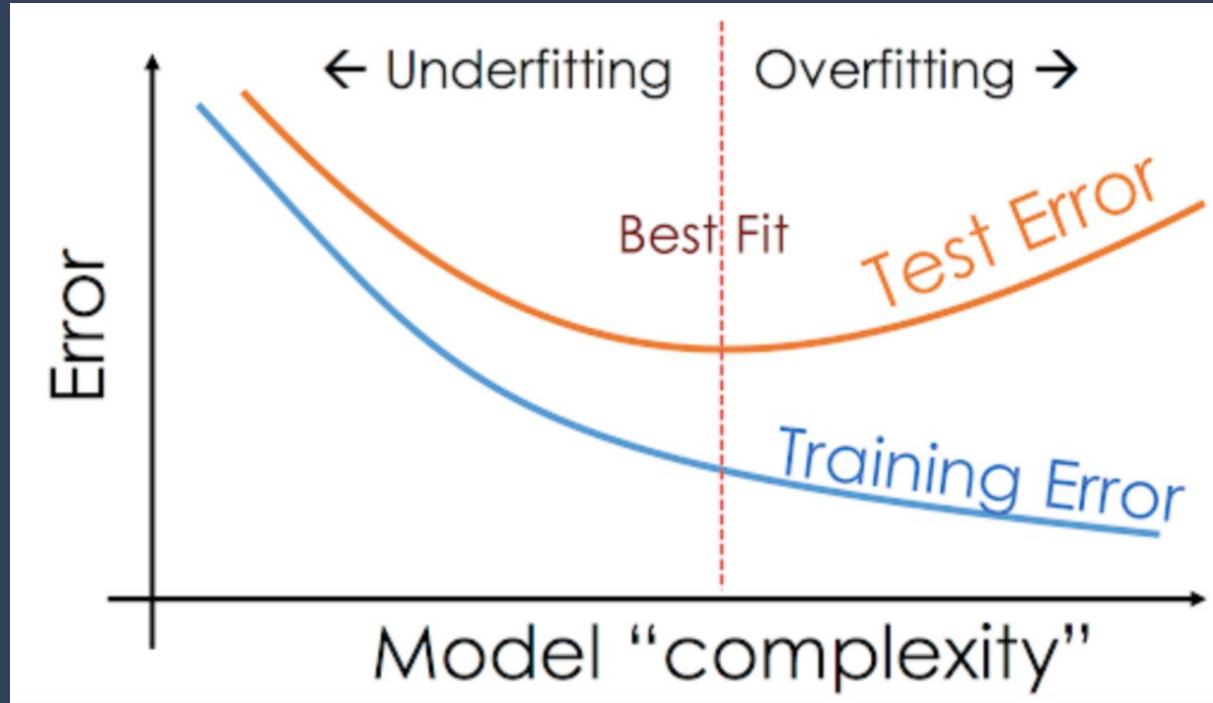
How can one determine if overfitting is occurring vs underfitting

- Training Loss vs. Validation Loss
- Model Complexity
 - Overfitting is more likely to occur when the model is too complex or has too many parameters relative to the size of the training dataset.
- Validation Metrics
- Training Duration

Training Loss vs. Validation Loss



Model Complexity



Batch Size in Neural Networks

Recall

$$\theta := \theta + \alpha \sum_{i=1}^n (y^{(i)} - h_{\theta}(x^{(i)})) x^{(i)}$$

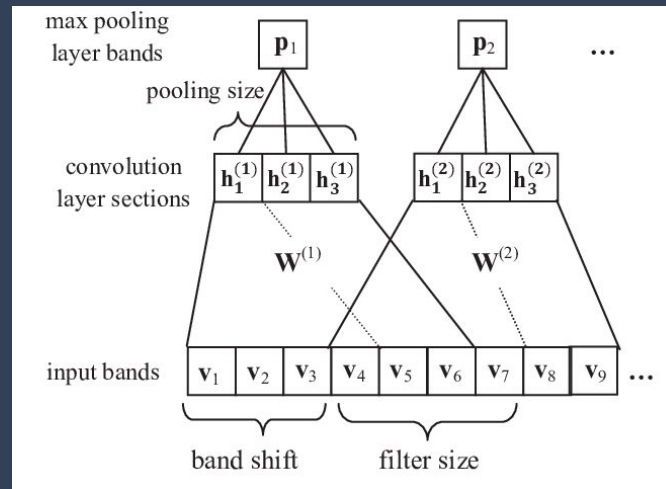
Batch Gradient Descent - Looks at every example in the entire training set on every step

$$\theta := \theta + \alpha (y^{(i)} - h_{\theta}(x^{(i)})) x^{(i)}$$

Stochastic Gradient Descent - we repeatedly run through the training set and update the parameters for each data point

Weight Sharing in CNN and RNN

- Weight sharing is a concept used in both Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs) in the field of deep learning.
- It refers to the practice of using the same set of parameters (weights and biases) for multiple units or neurons in a neural network layer.
- This technique has advantages in terms of reducing the number of parameters in the model and can help capture certain patterns and relationships in data.
 - In CNNs, weight sharing is primarily used in convolutional layers.
 - In RNNs, weight sharing is applied across the recurrent connections at each time step.



Key Takeaways

- Deep Learning Concepts
- Feedforward Neural Networks
- Convolutional Neural Networks
- Recurrent Neural Networks

